
Documentação de Projeto

para o sistema

CoinPrimer

Versão 1.0

Projeto de sistema elaborado pelo aluno Matheus Felipe
como parte da disciplina **Projeto de Software**.

13/05/2026

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Matheus Felipe	13/05/2026	Criação inicial do documento com diagramas UML (casos de uso, classes, componentes, sequência, estados, comunicação) e modelagem de dados.	1.0
Matheus Felipe	27/05/2026	Inserção de histórias de usuário e códigos em plantuml.	2.0

1. Introdução

Este documento agrega: a elaboração e revisão de modelos de domínio e modelos de projeto para o sistema **CoinPremier**. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

O **CoinPremier** é um sistema de moeda virtual acadêmica que permite aos professores reconhecerem seus alunos através de moedas digitais, que podem ser trocadas por produtos e descontos oferecidos por empresas parceiras.

Objetivos do sistema:

- Incentivar o bom desempenho acadêmico e o engajamento dos alunos
- Fortalecer a parceria entre instituições de ensino e empresas
- Gamificar a experiência universitária
- Reconhecer publicamente o mérito estudantil

Domínio do sistema: O sistema envolve 4 tipos de atores (Aluno, Professor, Empresa Parceira e Administrador), vinculados a instituições de ensino, onde professores recebem 1.000 moedas virtuais por semestre e distribuem aos alunos como reconhecimento. Os alunos acumulam essas moedas e podem trocá-las por vantagens reais (produtos, descontos) oferecidas por empresas parceiras através de uma lojinha virtual.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

O sistema CoinPremier envolve quatro atores humanos e um ator sistema automático:

2.1.1 Admin (Administrador)

- **Descrição:** Usuário com permissões totais sobre o sistema. Responsável pela gestão operacional.
- **Função principal:** Gerenciar instituições, cadastrar professores, gerenciar empresas parceiras, definir categorias de vantagens e auditar o sistema.
- **Relacionamento com o sistema:** Acesso privilegiado a todas as entidades.

2.1.2 Aluno

- **Descrição:** Estudante vinculado a uma instituição de ensino parceira.
- **Função principal:** Receber reconhecimentos (moedas) dos professores, navegar na lojinha virtual, favoritar, adicionar ao carrinho e resgatar vantagens.
- **Relacionamento com o sistema:** Auto-cadastrado; possui saldo acumulativo.

2.1.3 Professor

- **Descrição:** Docente pré-cadastrado pelo admin, vinculado a uma instituição de ensino.
- **Função principal:** Distribuir moedas aos alunos como reconhecimento por mérito acadêmico, participação, criatividade, liderança etc.
- **Relacionamento com o sistema:** Recebe 1.000 moedas automaticamente a cada semestre (acumuláveis).

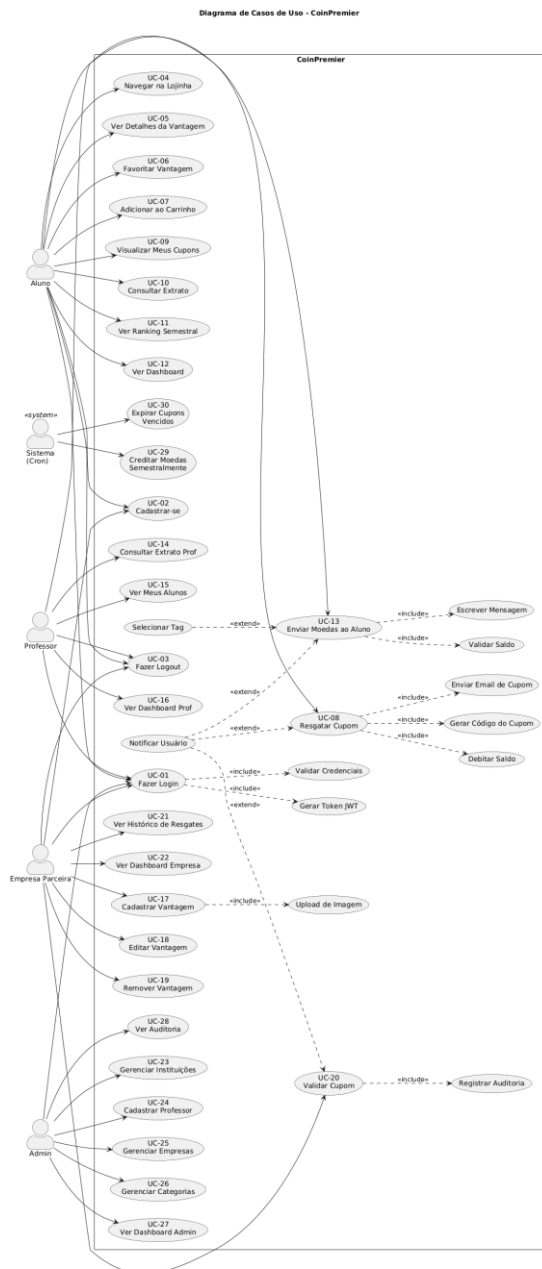
2.1.4 Empresa Parceira

- **Descrição:** Empresa que oferece vantagens (produtos, descontos) aos alunos.
- **Função principal:** Cadastrar vantagens, receber notificações de resgates e validar cupons apresentados presencialmente pelos alunos.
- **Relacionamento com o sistema:** Auto-cadastrada; gerencia seu próprio catálogo.

2.1.5 Sistema (Ator Automático)

- **Descrição:** Processos automáticos executados pelo sistema sem intervenção humana.
- **Função principal:** Executar jobs agendados, como crédito semestral de moedas (dia 1º do primeiro mês de cada semestre) e expiração automática de cupons vencidos.
- **Relacionamento com o sistema:** Cron jobs agendados via node-cron.

2.2 Modelo de Casos de Uso e Histórias de usuário



Aluno

- Como aluno, eu quero realizar meu autocadastro no sistema (vinculado a uma instituição de ensino parceira), para ter acesso à plataforma.
- Como aluno, eu quero receber moedas dos professores, para acumular saldo como forma de reconhecimento pelo meu mérito acadêmico.

- Como aluno, eu quero navegar na lojinha virtual, favoritar itens e adicioná-los ao carrinho, para gerenciar os produtos e descontos do meu interesse.
- Como aluno, eu quero resgatar vantagens utilizando meu saldo de moedas (Caso de Uso UC-08), para gerar um código de cupom com validade e utilizá-lo na empresa parceira.

Professor

- Como professor, eu quero receber um crédito automático de 1.000 moedas virtuais a cada semestre, para ter recursos para incentivar os estudantes.
- Como professor, eu quero enviar moedas a um aluno (Caso de Uso UC-13), incluindo uma quantidade, uma mensagem e uma tag de reconhecimento (como mérito acadêmico, liderança, criatividade, etc.), para reconhecer publicamente o desempenho dele.

Empresa Parceira

- Como empresa parceira, eu quero realizar o autocadastro no sistema, para gerenciar meu próprio catálogo de ofertas.
- Como empresa parceira, eu quero cadastrar vantagens (produtos e descontos), para disponibilizá-las aos alunos na lojinha virtual.
- Como empresa parceira, eu quero validar os cupons (Caso de Uso UC-20) apresentados presencialmente pelos alunos, para alterar o status do cupom para "Utilizado" e registrar uma auditoria no sistema.
- Como empresa parceira, eu quero receber notificações de resgates realizados, para ter controle sobre a emissão de cupons de minhas vantagens.

Administrador (Admin)

- Como administrador, eu quero ter acesso privilegiado para gerenciar instituições de ensino e cadastrar professores, para permitir que o ciclo de reconhecimento acadêmico se inicie.
- Como administrador, eu quero gerenciar empresas parceiras e definir categorias de vantagens, para organizar o catálogo oferecido aos alunos.
- Como administrador, eu quero poder auditar e gerenciar todo o sistema operacionalmente (inclusive bloqueando/desbloqueando usuários), para garantir o bom funcionamento da plataforma.

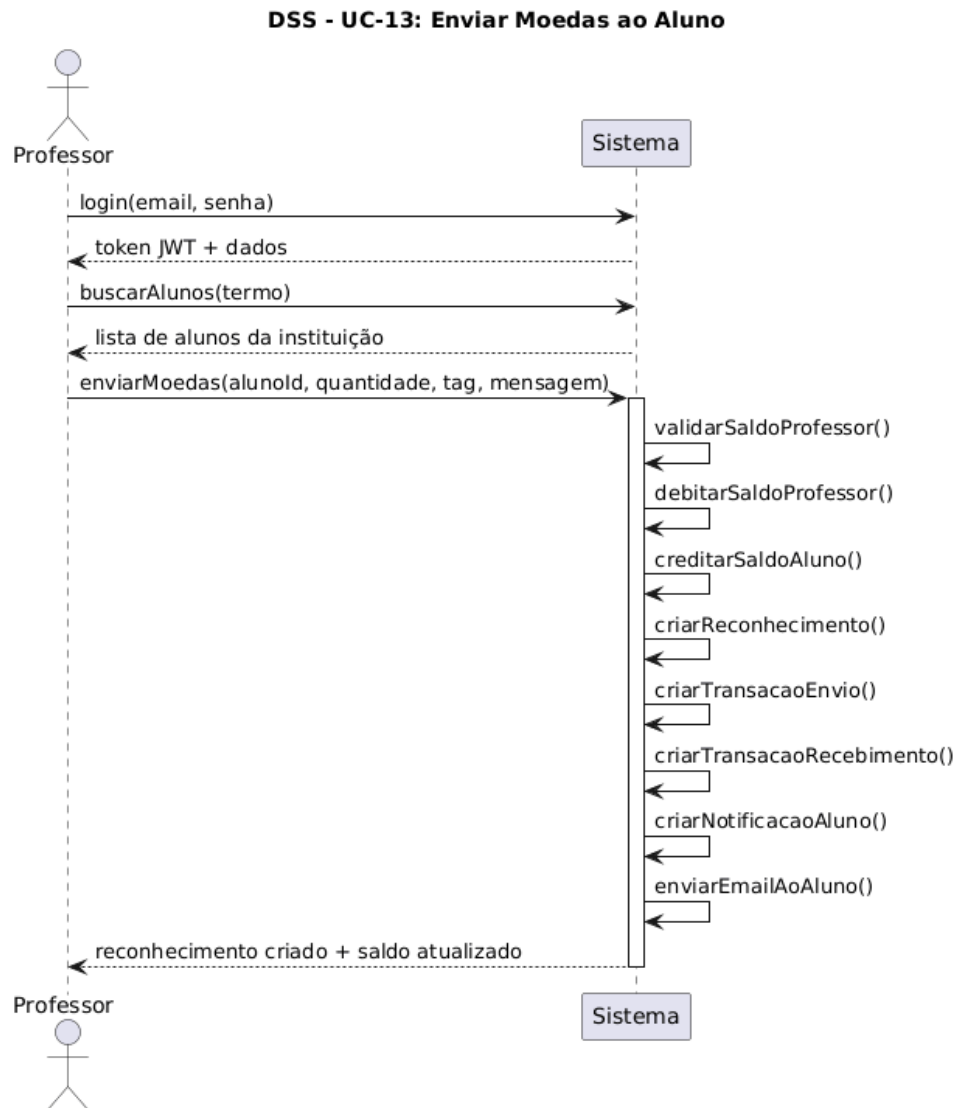
Sistema (Ator Automático)

- Como sistema automático, eu preciso executar rotinas (jobs agendados) para creditar as moedas semestrais aos professores no dia 1º do primeiro mês de cada semestre.
- Como sistema automático, eu preciso expirar automaticamente os cupons que ultrapassarem sua data de validade, para que eles não possam mais ser utilizados.

2.3 Diagrama de Sequência do Sistema

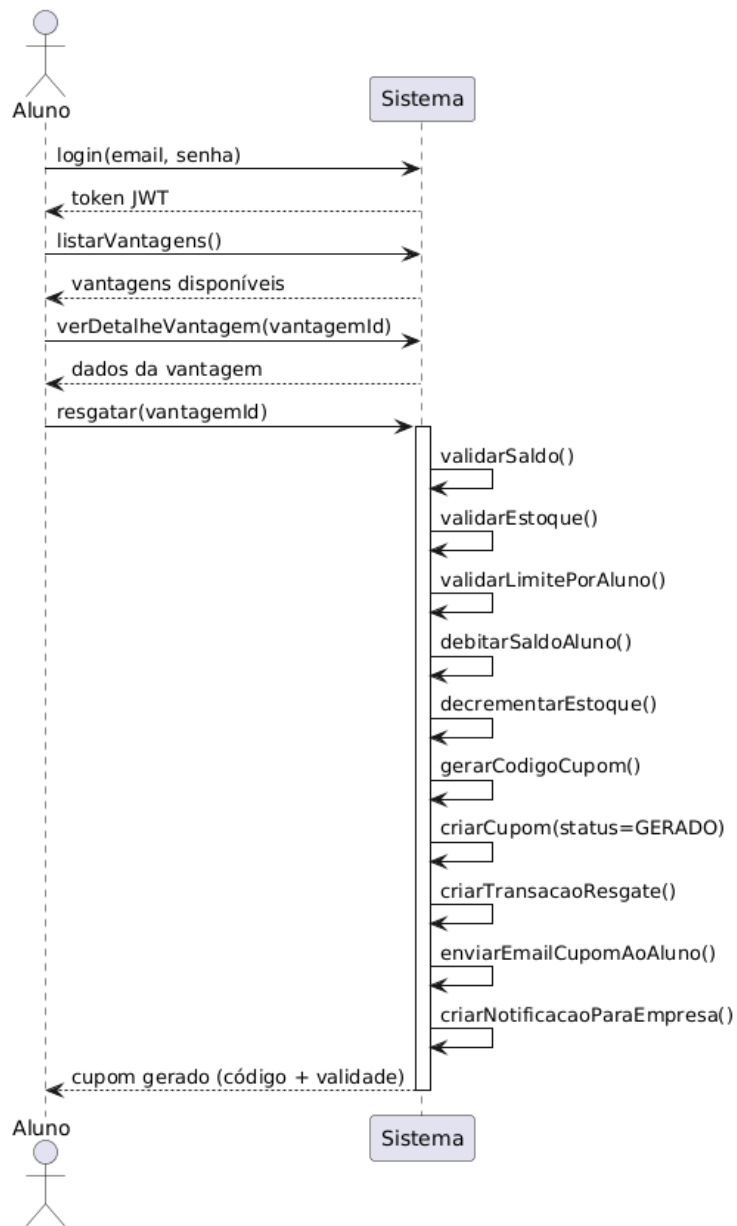
DSS-01: Enviar Moedas (UC-13)	
Contrato	CO-01
Operação	enviarMoedas(professorId, alunoId, quantidade, tag, mensagem)

Referências cruzadas	UC-13 (Enviar Moedas)
Pré-condições	1. Professor autenticado com role = PROFESSOR 2. Aluno existe e pertence à mesma instituição do professor 3. Saldo do professor $\geq$ quantidade 4. Quantidade é inteiro positivo ≥ 1 5. Mensagem possui entre 10 e 500 caracteres 6. Tag pertence ao enum TagReconhecimento
Pós-condições	1. Professor.saldoMoedas foi decrementado em quantidade 2. Aluno.saldoMoedas foi incrementado em quantidade 3. Uma instância de Reconhecimento foi criada 4. Duas instâncias de Transação foram criadas (ENVIO e RECEBIMENTO) 5. Uma Notificação foi criada para o aluno 6. Um e-mail foi disparado ao aluno 7. Toda a operação foi executada atomicamente (tudo ou nada)

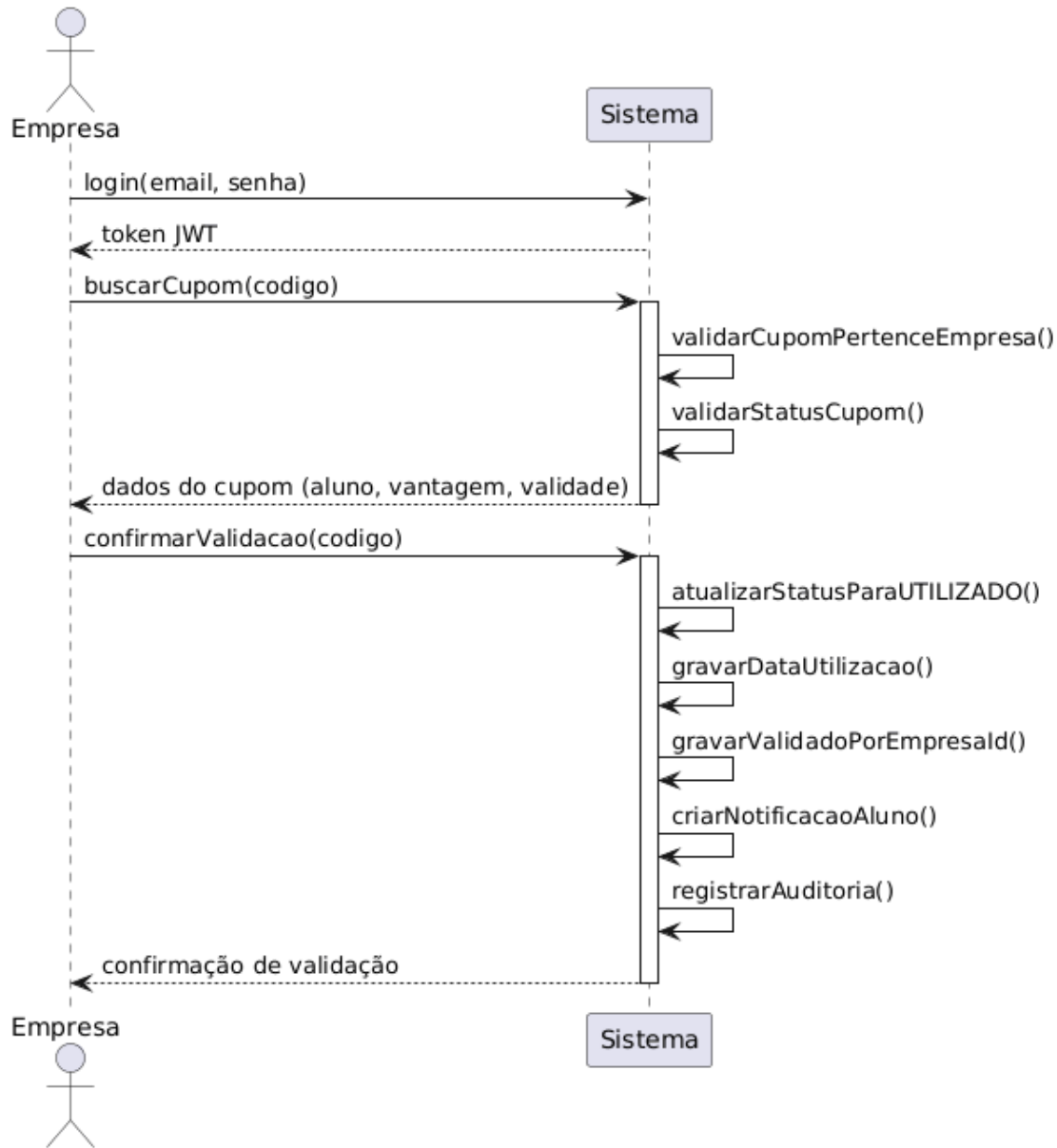


DSS-02: Resgatar Cupom (UC-08)	
Contrato	CO-02
Operação	resgatarVantagem(alunoId, vantagemId)
Referências cruzadas	UC-08 (Resgatar Cupom)
Pré-condições	1. Aluno autenticado com role = ALUNO 2. Vantagem existe e está com ativo = true 3. Saldo do aluno $\geq$ custoMoedas da vantagem 4. Estoque da vantagem > 0 (ou ilimitado/null) 5. Aluno não atingiu limitePorAluno para esta vantagem
Pós-condições	1. Aluno.saldoMoedas foi decrementado em custoMoedas 2. Vantagem.estoque foi decrementado em 1 (se não for ilimitado) 3. Uma instância de Cupom foi criada com status = GERADO, código único e custoMoedasSnapshot travado 4. Cupom.dataValidade foi calculada como now() + validadeCupomDias 5. Uma instância de Transacao foi criada (RESGATE) 6. Um email com o cupom foi enviado ao aluno 7. Uma Notificacao foi criada para a empresa 8. Toda a operação foi executada atomicamente

DSS - UC-08: Resgatar Cupom



DSS-03: Validar Cupom (UC-20)	
Contrato	CO-03
Operação	validarCupom(empresaId, codigoCupom)
Referências cruzadas	UC-20 (Validar Cupom)
Pré-condições	1. Empresa autenticada com role = EMPRESA 2. Cupom existe com o código informado 3. Cupom.status = GERADO 4. Cupom.dataValidade > now() 5. Cupom pertence a uma vantagem da empresa logada (Cupom.vantagem.empresaId = empresa.id)
Pós-condições	1. Cupom.status foi alterado para UTILIZADO 2. Cupom.dataUtilizacao foi preenchido com now() 3. Cupom.validadoPorEmpresaId recebeu o id da empresa validadora (auditoria) 4. Uma Notificacao foi criada para o aluno5. Um registro de auditoria foi gerado

DSS - UC-20: Validar Cupom

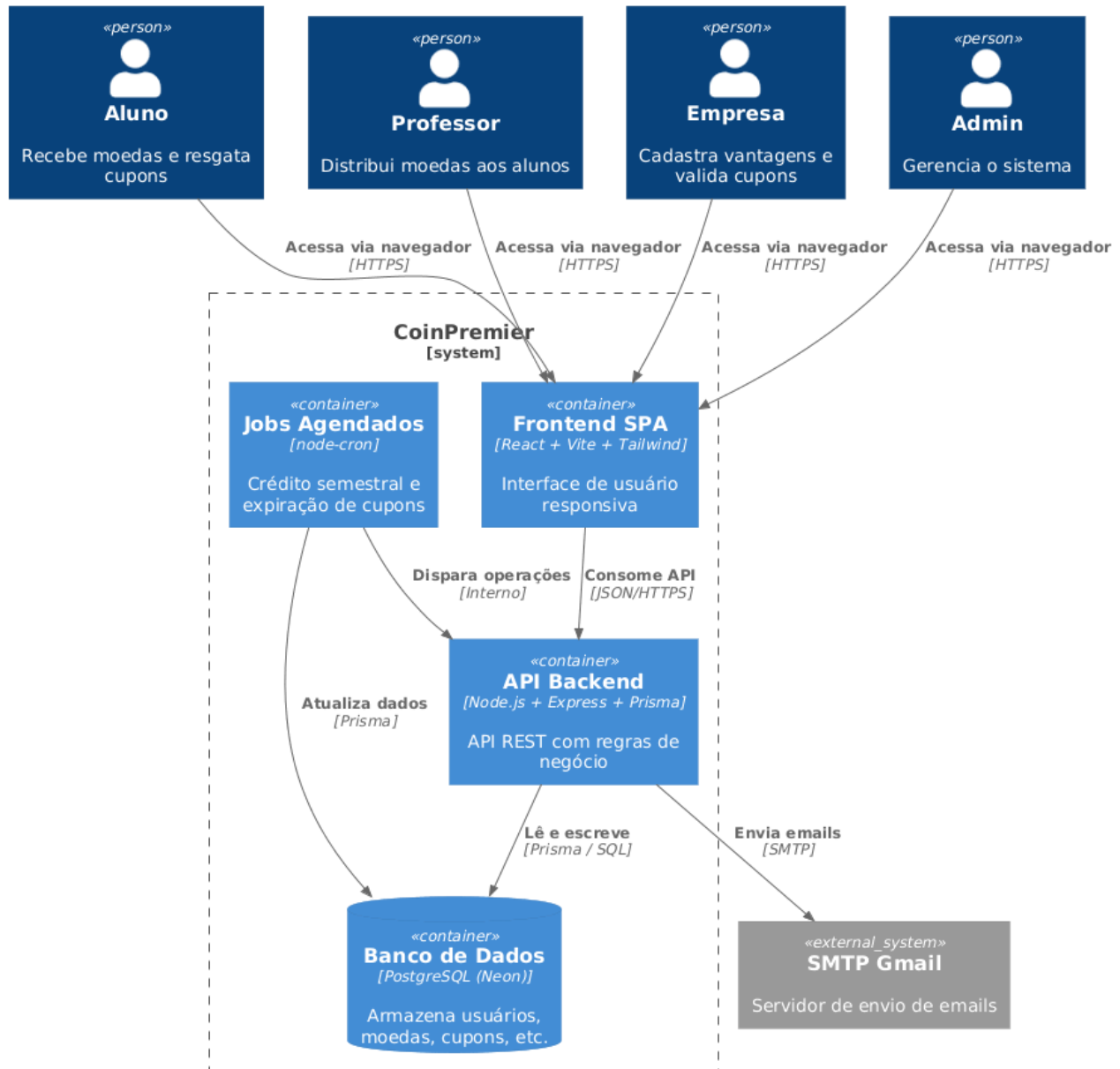
3. Modelos de Projeto

3.1 Arquitetura

O CoinPrimer segue arquitetura em camadas (MVC) no backend e Component-Based no frontend, com separação clara de responsabilidades:

- **Frontend:** React + Vite + Tailwind CSS + Zustand
- **Backend:** Node.js + Express + Prisma ORM (MVC + Repository Pattern)
- **Banco:** PostgreSQL hospedado no Neon Database
- **Comunicação:** API REST com autenticação JWT
- **Emails:** SMTP via Nodemailer

Diagrama de Containers (C4) - CoinPremier



3.2 Diagrama de Componentes e Implantação.

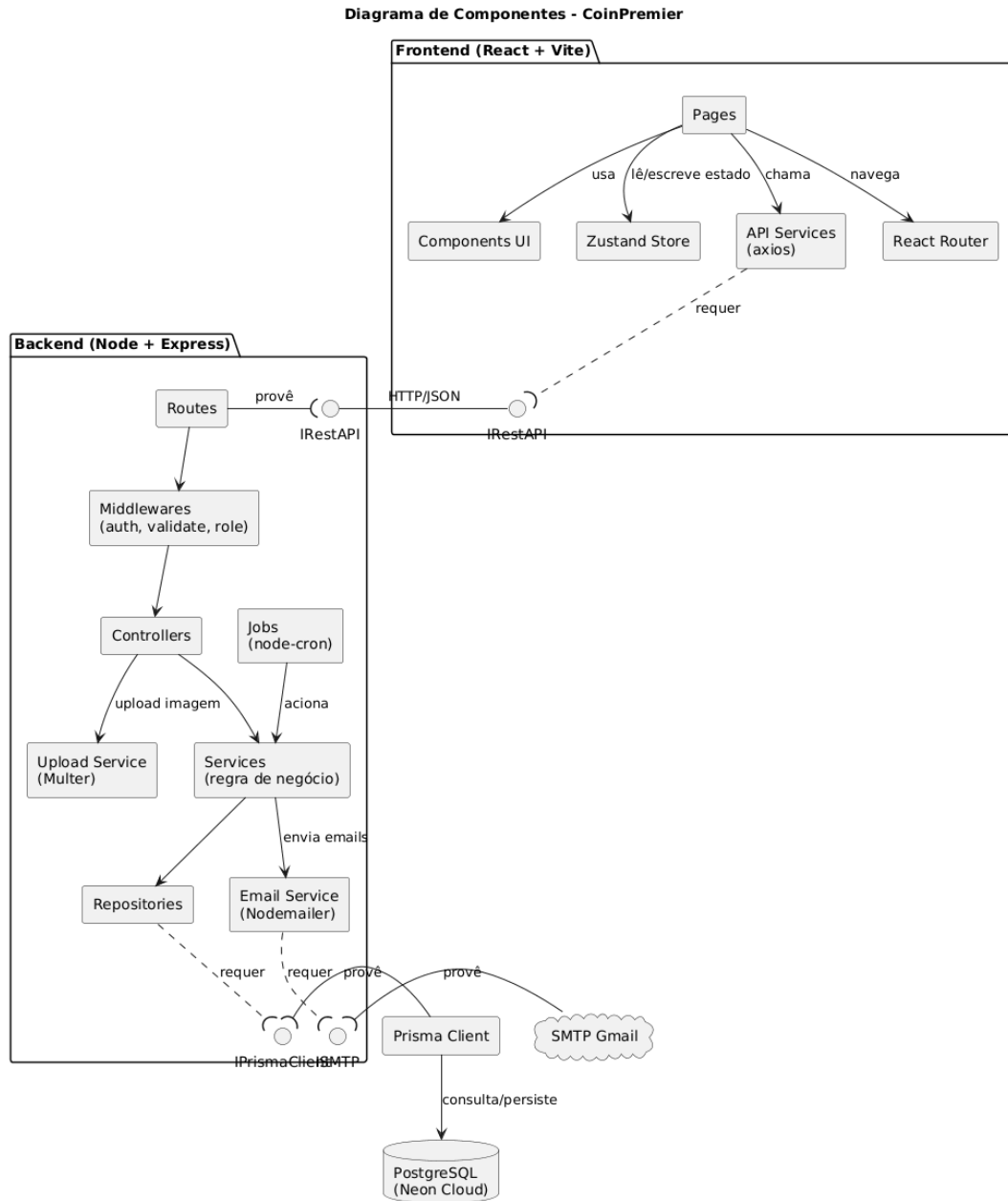
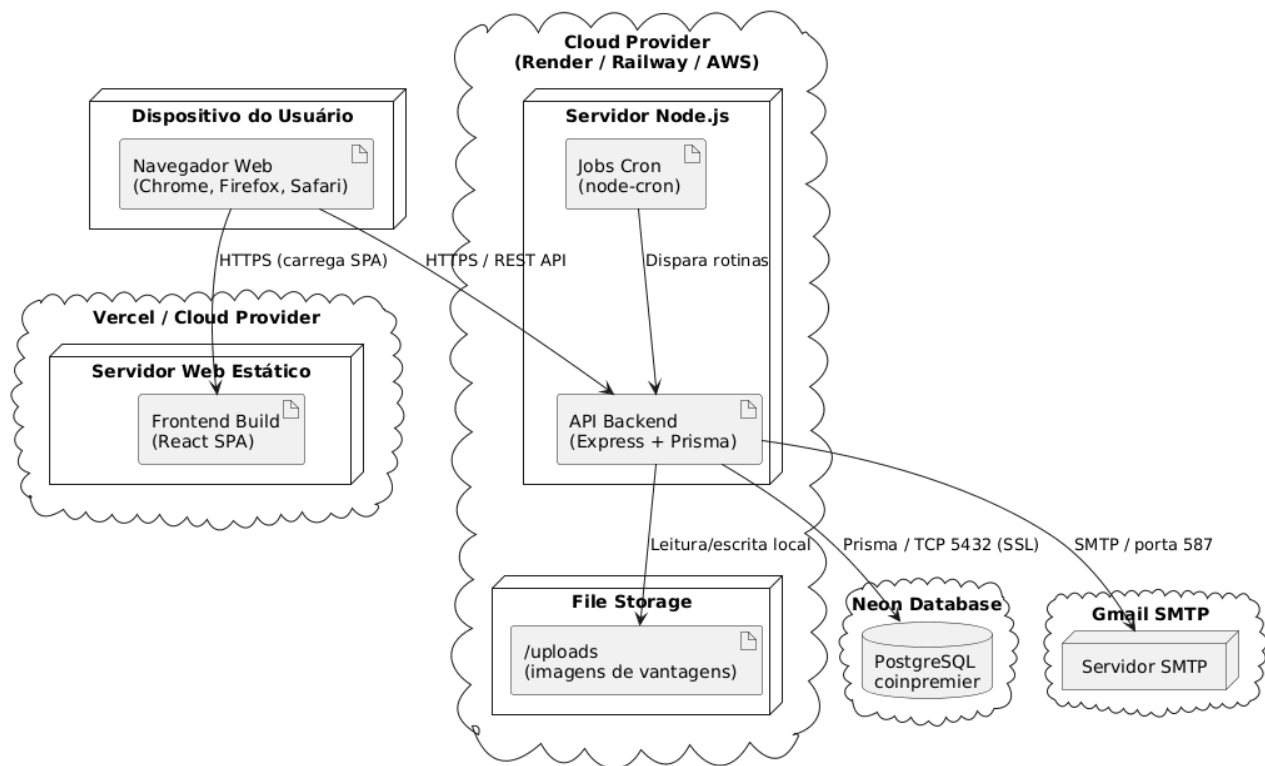
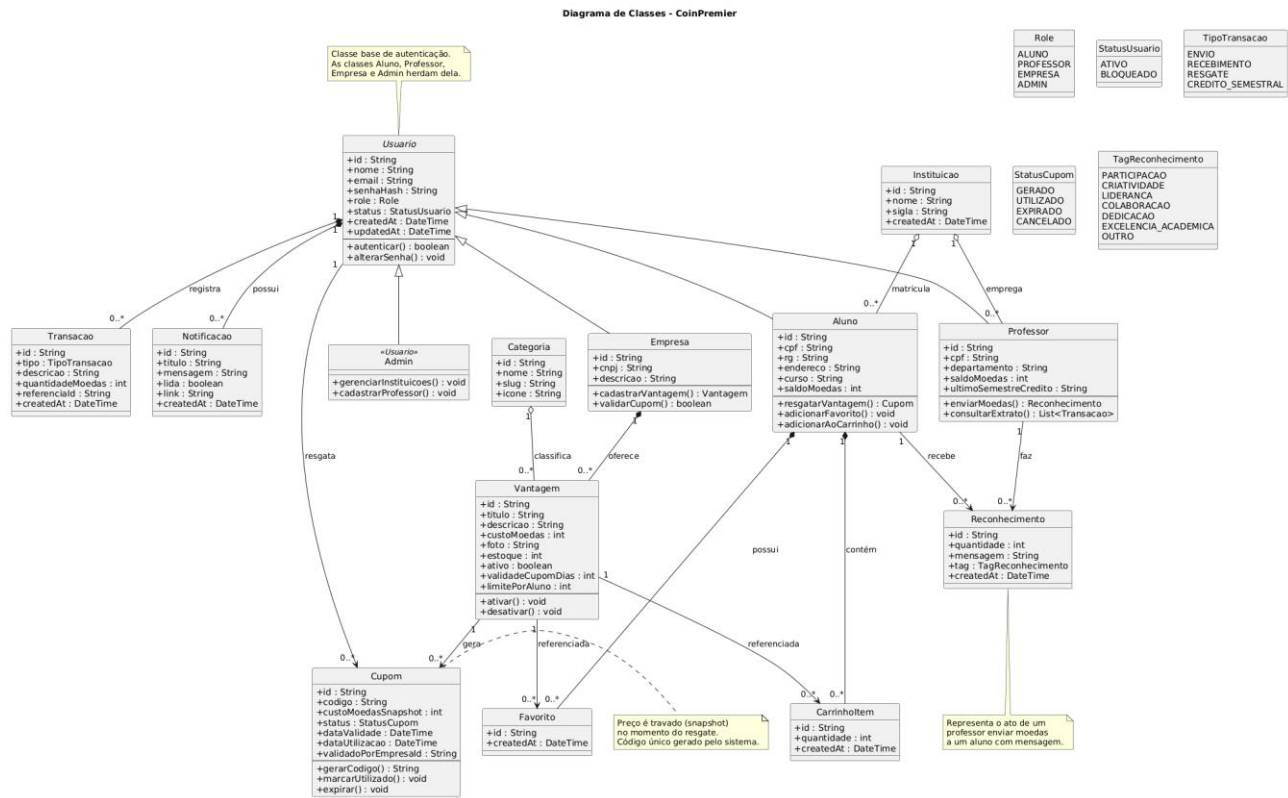


Diagrama de Implantação - CoinPremier

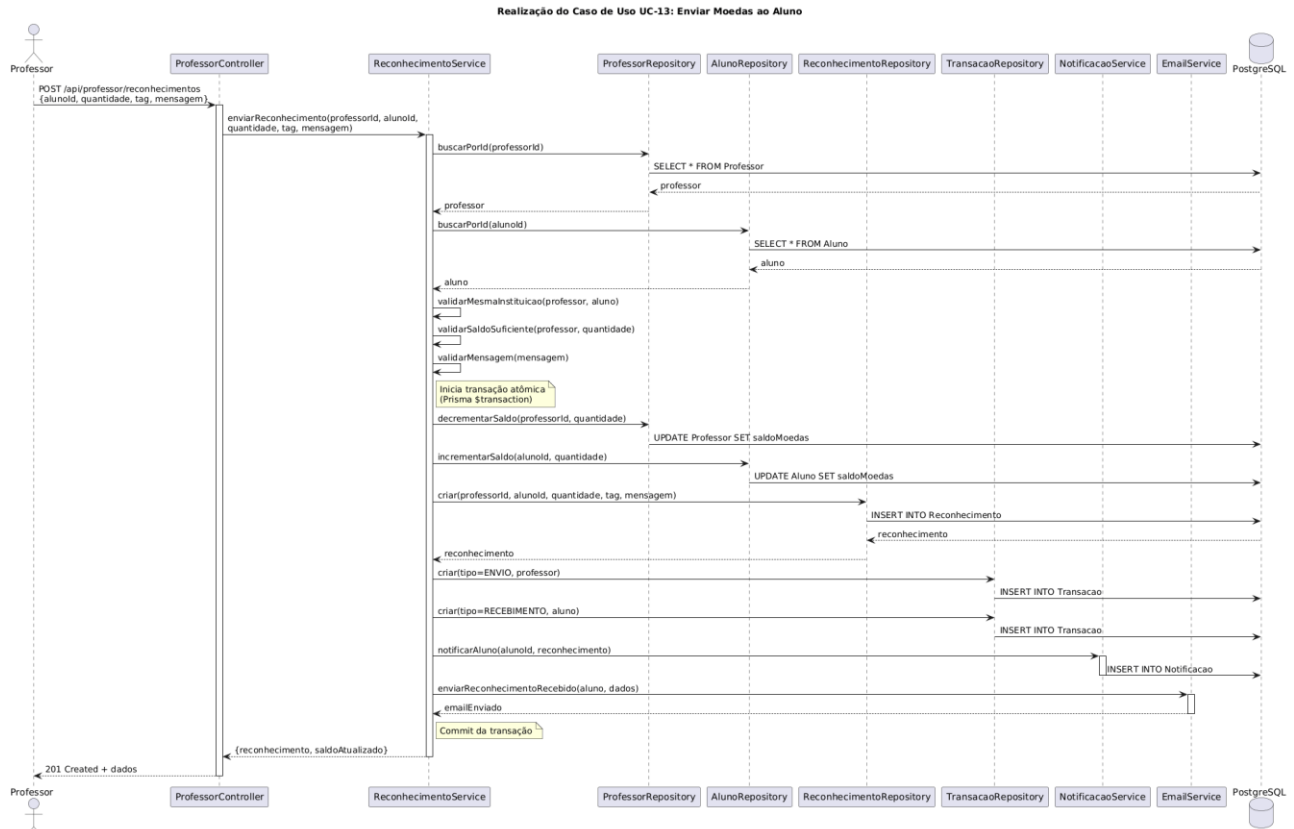


3.3 Diagrama de Classes

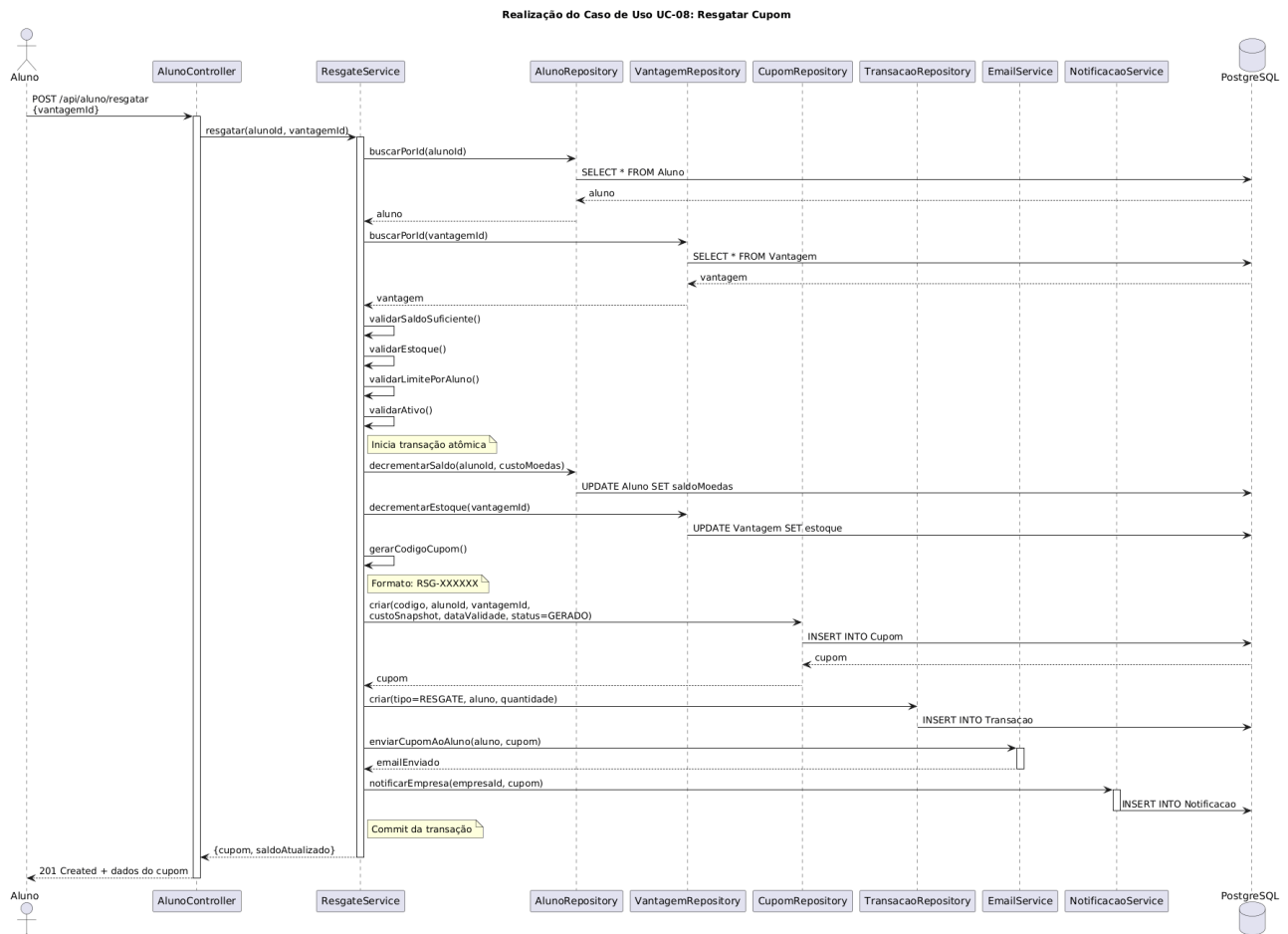


3.4 Diagramas de Sequência

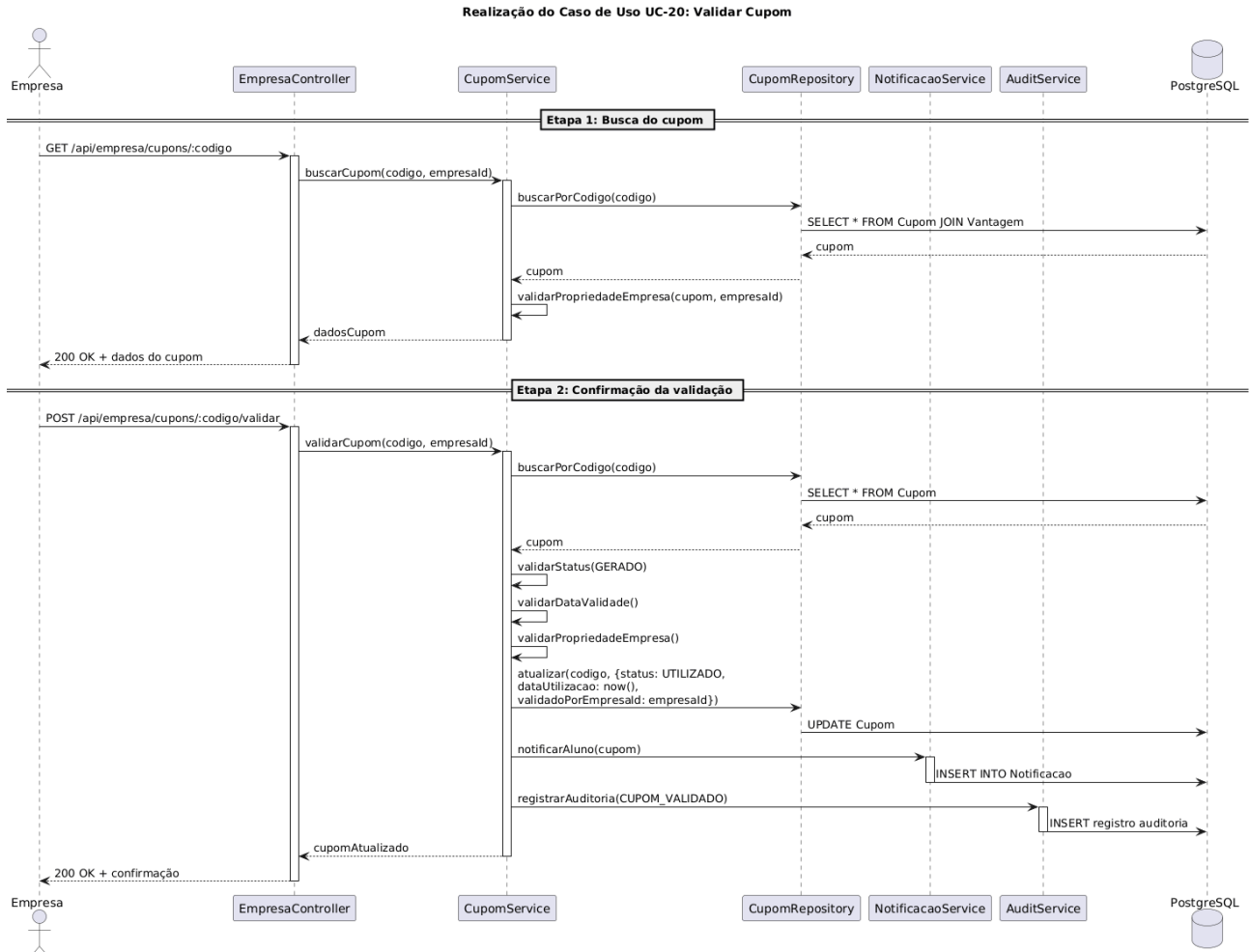
3.4.1 DS-01: Realização do UC-13 (Enviar Moedas ao Aluno)



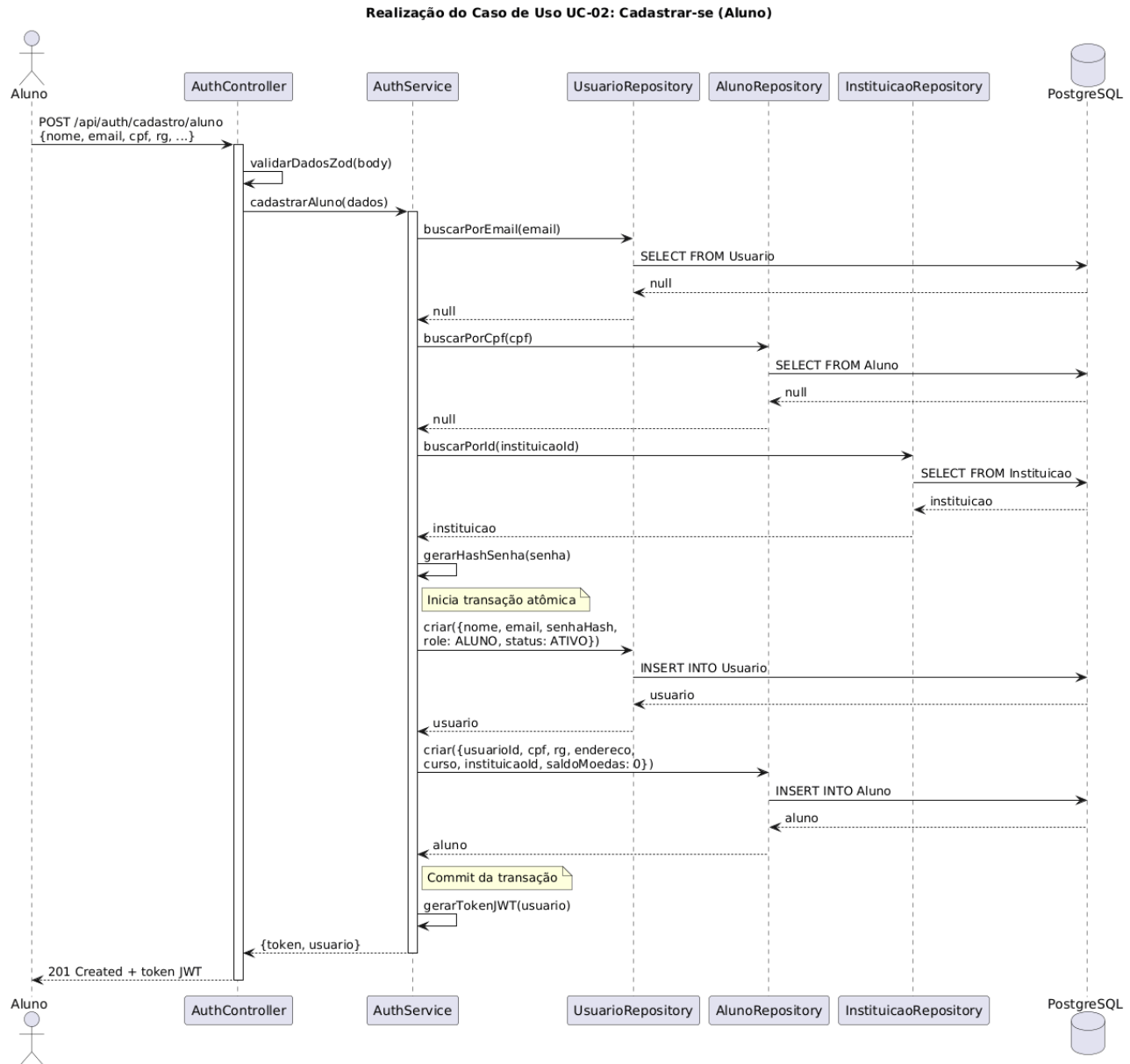
3.4.2 DS-02: Realização do UC-08 (Resgatar Cupom)



3.4.3 DS-03: Realização do UC-20 (Validar Cupom)



3.4.4 DS-04: Realização do UC-02 (Cadastrar Aluno)

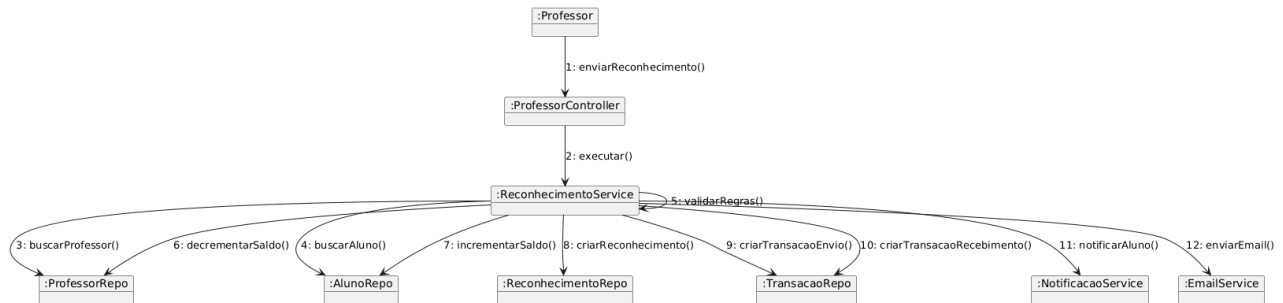


3.5 Diagramas de Comunicação

Os diagramas de comunicação representam as mesmas interações dos diagramas de sequência, mas com ênfase nas ligações entre objetos (visão estrutural).

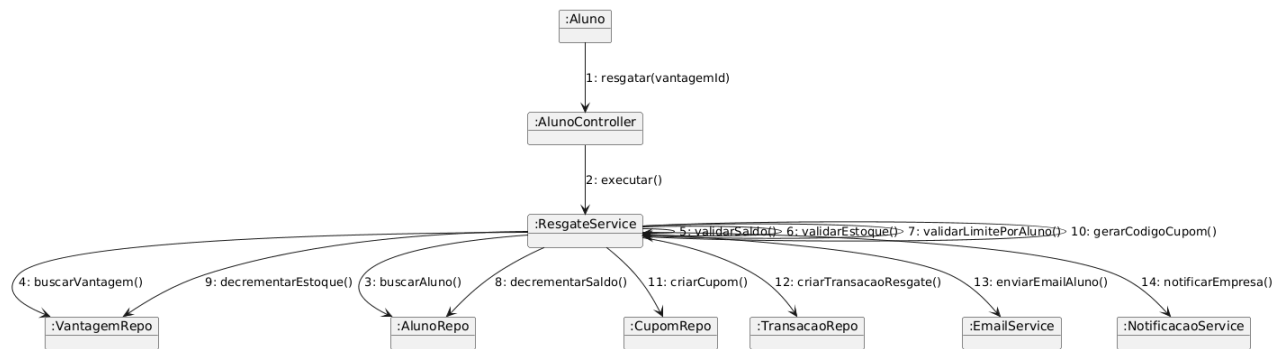
3.5.1 DC-01: Realização do UC-13 (Enviar Moedas)

Diagrama de Comunicação - UC-13 Enviar Moedas



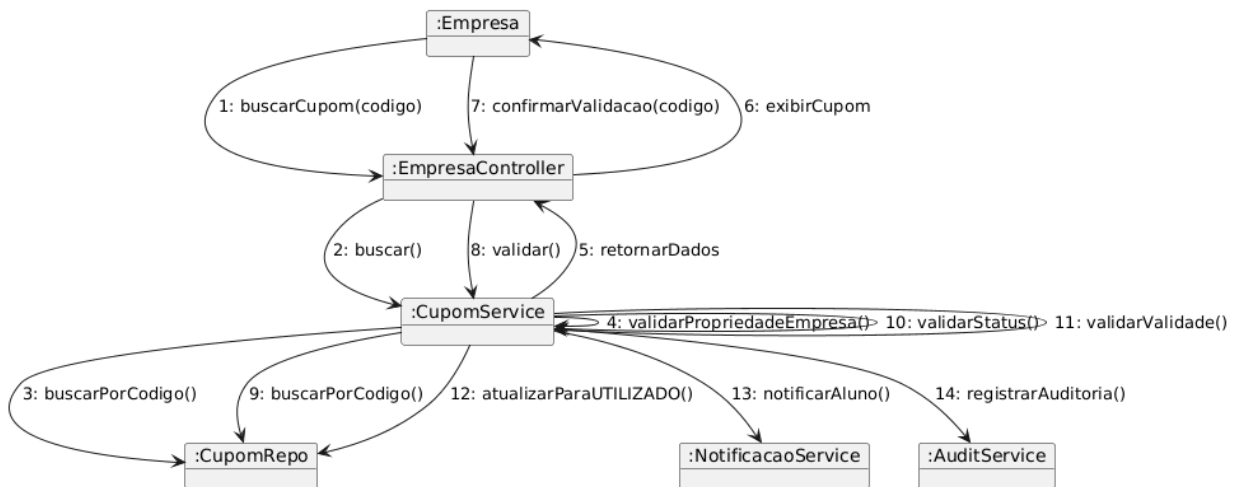
3.5.2 DC-02: Realização do UC-08 (Resgatar Cupom)

Diagrama de Comunicação - UC-08 Resgatar Cupom



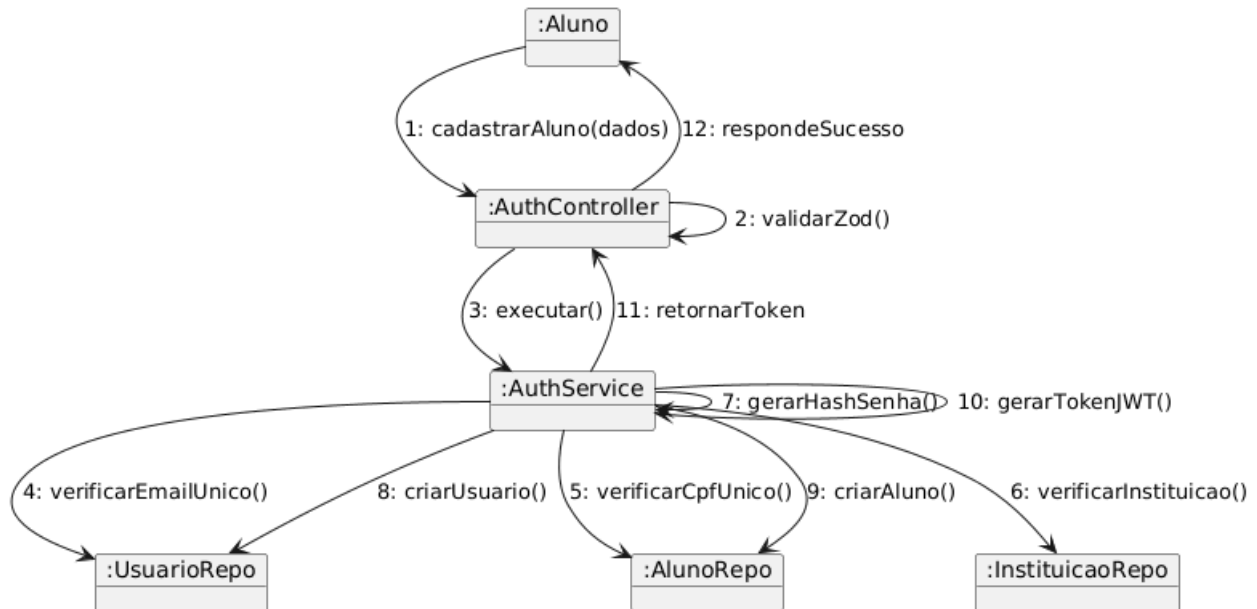
3.5.3 DC-03: Realização do UC-20 (Validar Cupom)

Diagrama de Comunicação - UC-20 Validar Cupom



3.5.4 DC-04: Realização do UC-02 (Cadastrar Aluno)

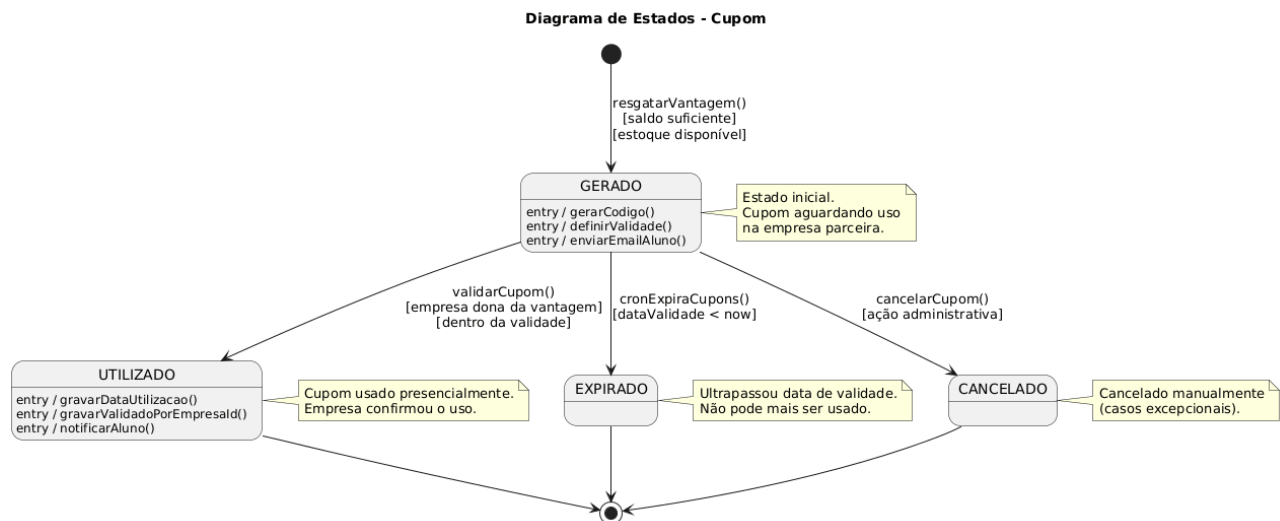
Diagrama de Comunicação - UC-02 Cadastrar-se (Aluno)



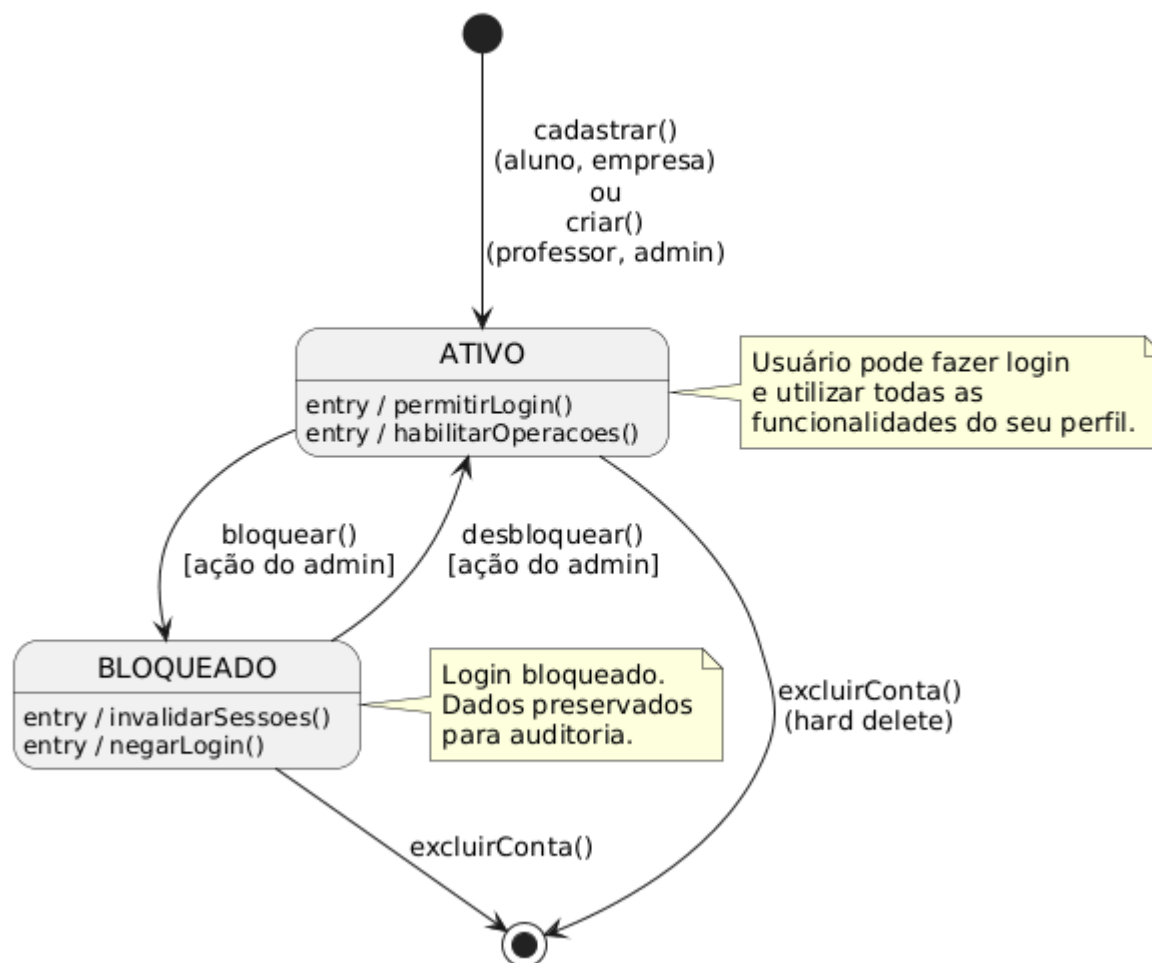
3.6 Diagramas de Estados

Nesta subseção são apresentados os diagramas de estados das principais entidades do sistema, mostrando suas transições ao longo do ciclo de vida.

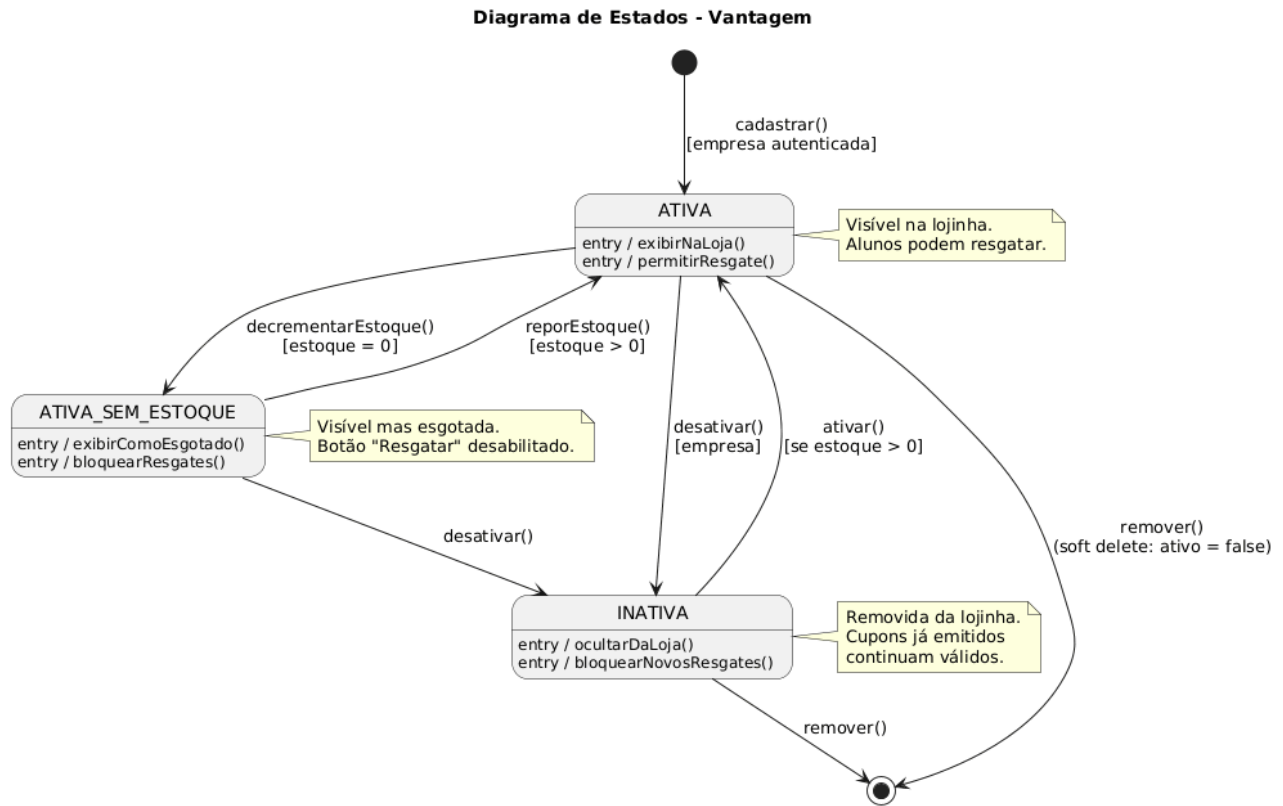
3.6.1 DE-01: Estados do Cupom



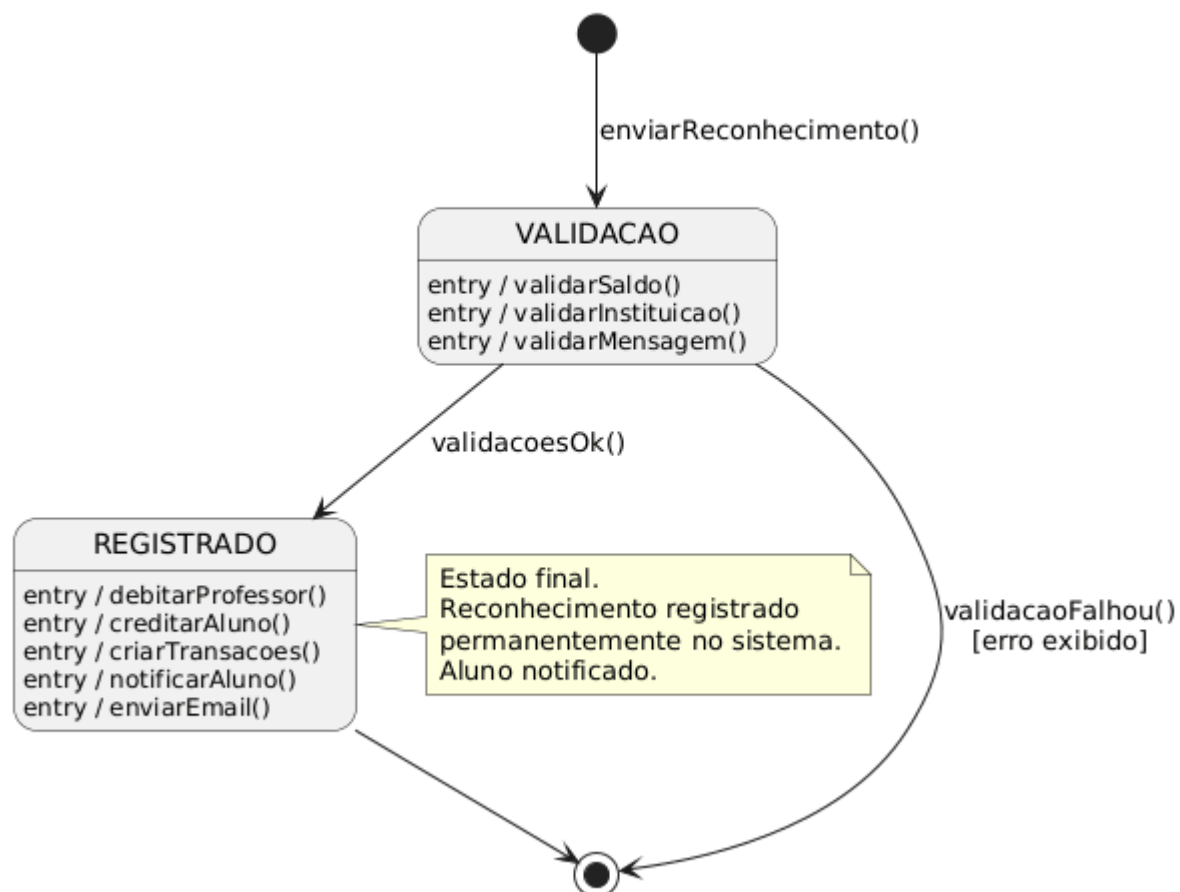
3.6.2 DE-02: Estados do Usuário

Diagrama de Estados - Usuário

3.6.3 DE-03: Estados da Vantagem

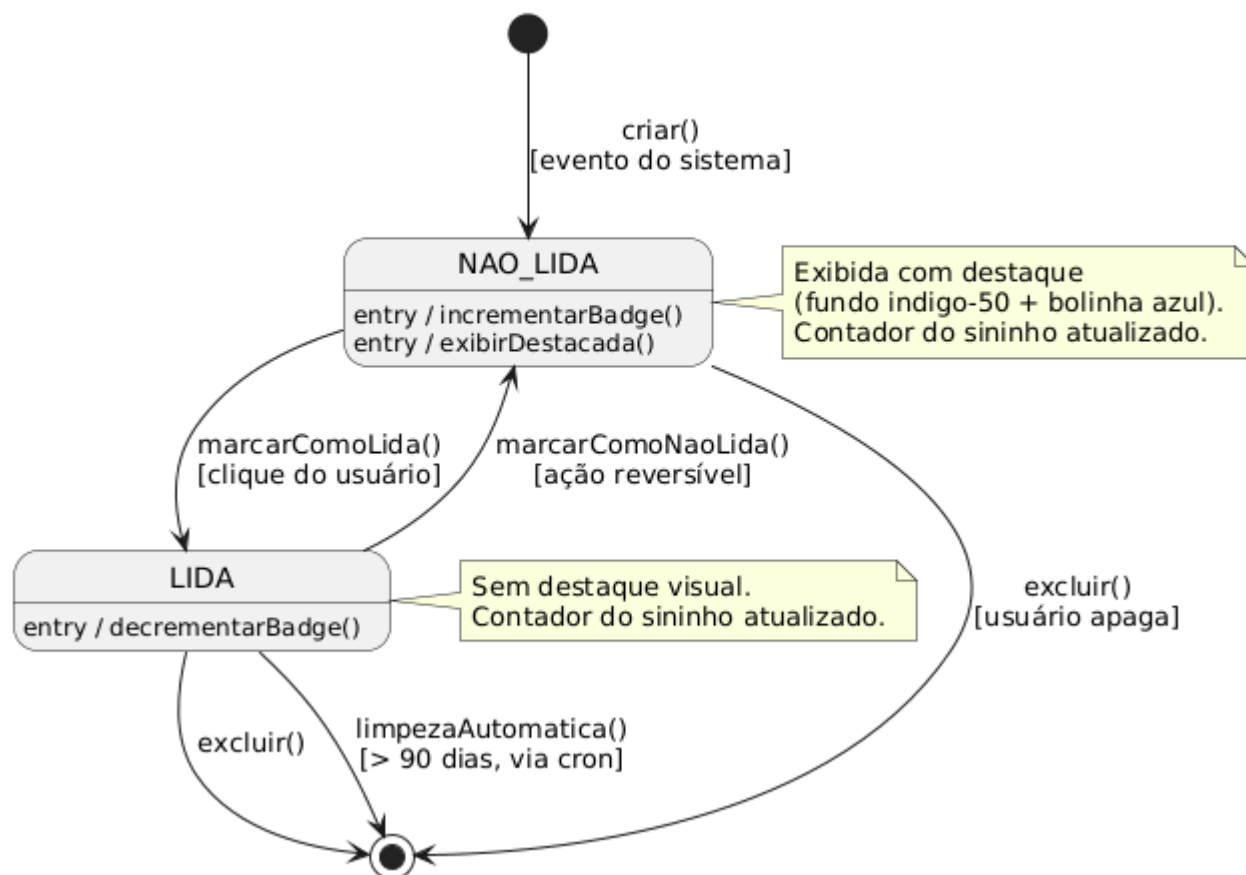


3.6.4 DE-04: Estados do Reconhecimento (simplificado)

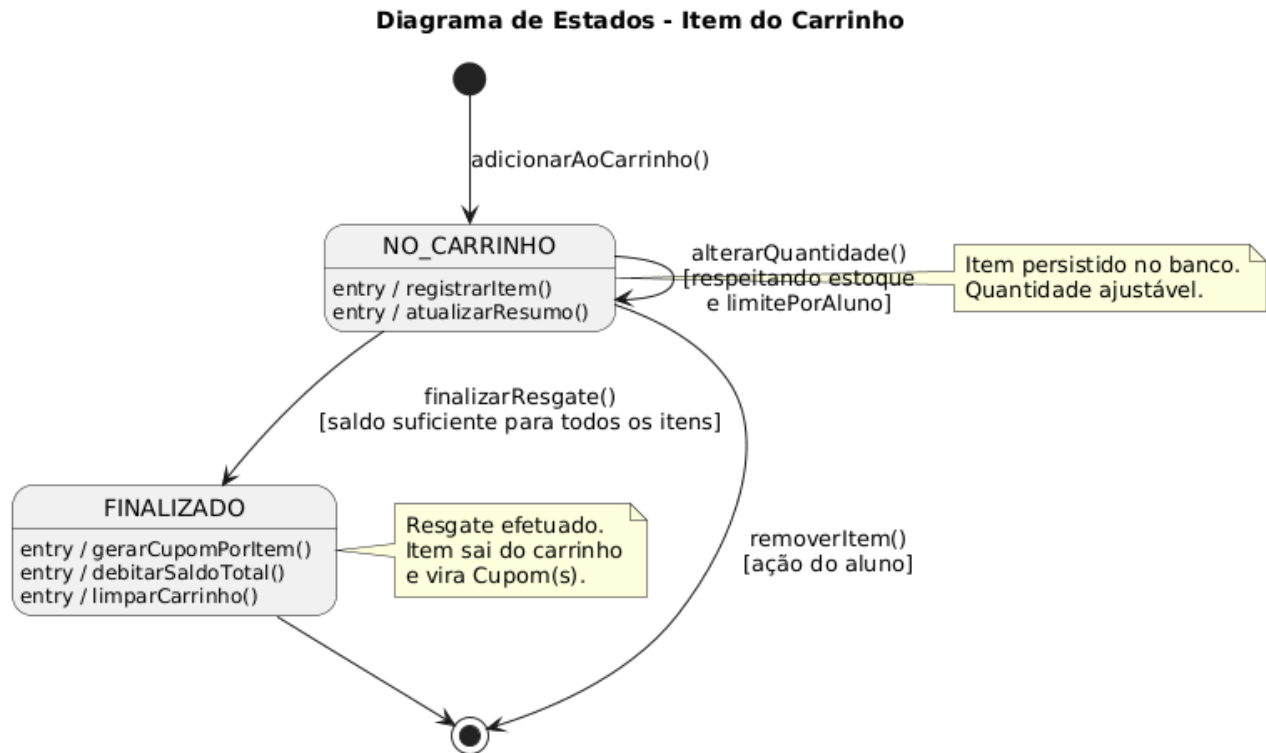
Diagrama de Estados - Reconhecimento

3.6.5 DE-05: Estados da Notificação

Diagrama de Estados - Notificação



3.6.6 DE-06: Estados do Item do Carrinho



4. Modelos de Dados

4.1 Estratégia de Mapeamento Objeto-Relacional

O CoinPremier utiliza a estratégia **ORM + Repository Pattern** para acesso ao banco de dados, implementada com **Prisma ORM** sobre **PostgreSQL** (hospedado no Neon Cloud).

Critério	Decisão	Justificativa
ORM	Prisma Client	Type-safety nativo, migrations automáticas, schema declarativo, excelente suporte ao PostgreSQL
Padrão de Acesso	Repository Pattern	Isola Services (lógica de negócio) da camada de persistência, facilita testes unitários (mocks) e permite futura troca de ORM
Transações Atômicas	prisma.\$transaction()	Garante ACID em operações críticas (resgate, envio de moedas)
IDs	CUID (@default(cuid()))	IDs únicos, ordenáveis e resistentes a colisão em sistemas distribuídos
Soft Delete	Flag ativo: boolean	Preserva integridade referencial (cupons emitidos permanecem válidos mesmo se vantagem for "removida")

4.1.2 Fluxo de Acesso aos Dados

Controller → Service → Repository → Prisma Client → PostgreSQL

- **Controller** recebe requisição HTTP e delega ao Service
- **Service** aplica regras de negócio e chama um ou mais Repositories
- **Repository** encapsula queries Prisma (nunca usado diretamente pelo Controller)
- **Prisma Client** traduz chamadas em SQL e gerencia conexões
- **PostgreSQL** persiste os dados

4.1.3 Técnicas Especiais de Mapeamento

Herança (Usuario → Aluno/Professor/Empresa): Modelada como **composição 1:1** no banco relacional, não como tabela única. Cada perfil tem sua tabela com chave estrangeira `usuarioId @unique`. A distinção entre perfis é feita pelo campo `role` em `Usuario`. Esta estratégia é conhecida como **Class Table Inheritance**.

Relacionamentos N:N: Implementados por tabelas associativas com índices únicos compostos:

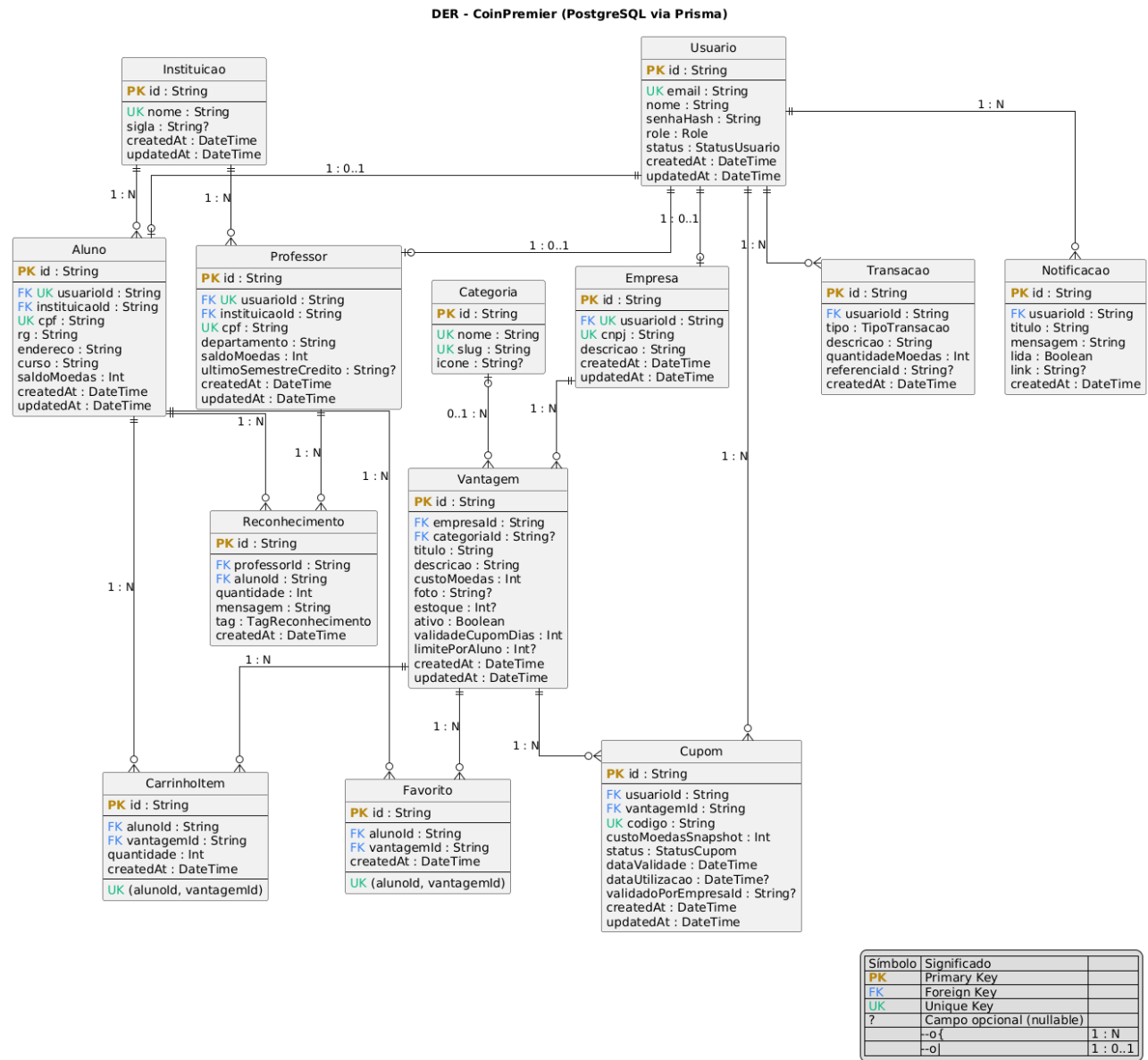
- Favorito (`alunoId`, `vantagemId`) — único composto
- CarrinhoItem (`alunoId`, `vantagemId`) — único composto
- Cupom e Reconhecimento — não usam `unique` composto pois podem ter múltiplos registros entre os mesmos pares

Snapshot de Valores: O campo `custoMoedasSnapshot` em `Cupom` guarda o preço no instante do resgate, garantindo imutabilidade histórica. Se a empresa alterar o preço da `Vantagem` posteriormente, cupons já emitidos mantêm o valor original.

Índices Compostos: Aplicados em consultas frequentes para otimização:

- `Transacao @@index([usuarioId, createdAt])` — extrato cronológico
- `Cupom @@index([status, dataValidade])` — expiração via cron
- `Reconhecimento @@index([alunoId, createdAt])` — histórico do aluno

4.2 Diagrama Entidade-Relacionamento (DER)



4.3 Esquema Físico do Banco (DDL Simplificado)

O script DDL abaixo representa a estrutura física gerada pelo Prisma Migrate. Apresentado em formato simplificado para fins de documentação.

```
sql
```

```
--
--
CREATE TYPE "Role" AS ENUM ('ALUNO', 'PROFESSOR', 'EMPRESA', 'ADMIN');
CREATE TYPE "StatusUsuario" AS ENUM ('ATIVO', 'BLOQUEADO');
CREATE TYPE "TipoTransacao" AS ENUM ('ENVIO', 'RECEBIMENTO', 'RESGATE', 'CREDITO_SEMESTRAL');
CREATE TYPE "StatusCupom" AS ENUM ('GERADO', 'UTILIZADO', 'EXPIRADO', 'CANCELADO');
CREATE TYPE "TagReconhecimento" AS ENUM (
```

```

'PARTICIPACAO',          'CRIATIVIDADE',          'LIDERANCA',          'COLABORACAO',
'DEDICACAO',              'EXCELENCIA_ACADEMICA',          'OUTRO'
);

--
CREATE TABLE "Usuario" (
  "id" TEXT PRIMARY KEY,
  "nome" TEXT NOT NULL,
  "email" TEXT UNIQUE NOT NULL,
  "senhaHash" TEXT NOT NULL,
  "role" TEXT NOT NULL,
  "status" "StatusUsuario" NOT NULL DEFAULT 'ATIVO',
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);

CREATE TABLE "Instituicao" (
  "id" TEXT PRIMARY KEY,
  "nome" TEXT UNIQUE NOT NULL,
  "sigla" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);

CREATE TABLE "Aluno" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT UNIQUE NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "instituicaoId" TEXT NOT NULL REFERENCES "Instituicao"("id"),
  "cpf" TEXT UNIQUE NOT NULL,
  "rg" TEXT NOT NULL,
  "endereco" TEXT NOT NULL,
  "curso" TEXT NOT NULL,
  "saldoMoedas" INTEGER NOT NULL DEFAULT 0,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Aluno_instituicaoId_idx" ON "Aluno"("instituicaoId");

CREATE TABLE "Professor" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT UNIQUE NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "instituicaoId" TEXT NOT NULL REFERENCES "Instituicao"("id"),
  "cpf" TEXT UNIQUE NOT NULL,
  "departamento" TEXT NOT NULL,
  "saldoMoedas" INTEGER NOT NULL DEFAULT 0,
  "ultimoSemestreCredito" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Professor_instituicaoId_idx" ON "Professor"("instituicaoId");

```

```

CREATE TABLE "Empresa" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT UNIQUE NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "cnpj" TEXT UNIQUE NOT NULL,
  "descricao" TEXT NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);

CREATE TABLE "Categoria" (
  "id" TEXT PRIMARY KEY,
  "nome" TEXT UNIQUE NOT NULL,
  "slug" TEXT UNIQUE NOT NULL,
  "icone" TEXT
);

CREATE TABLE "Vantagem" (
  "id" TEXT PRIMARY KEY,
  "empresaId" TEXT NOT NULL REFERENCES "Empresa"("id") ON DELETE CASCADE,
  "categoriaId" TEXT REFERENCES "Categoria"("id"),
  "titulo" TEXT NOT NULL,
  "descricao" TEXT NOT NULL,
  "custoMoedas" INTEGER NOT NULL,
  "foto" TEXT,
  "estoque" INTEGER,
  "ativo" BOOLEAN NOT NULL DEFAULT true,
  "validadeCupomDias" INTEGER NOT NULL DEFAULT 30,
  "limitePorAluno" INTEGER,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Vantagem_empresaId_idx" ON "Vantagem"("empresaId");
CREATE INDEX "Vantagem_categoriaId_ativo_idx" ON "Vantagem"("categoriaId", "ativo");

CREATE TABLE "Favorito" (
  "id" TEXT PRIMARY KEY,
  "alunoId" TEXT NOT NULL REFERENCES "Aluno"("id") ON DELETE CASCADE,
  "vantagemId" TEXT NOT NULL REFERENCES "Vantagem"("id") ON DELETE CASCADE,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE ("alunoId", "vantagemId")
);

CREATE TABLE "CarrinhoItem" (
  "id" TEXT PRIMARY KEY,
  "alunoId" TEXT NOT NULL REFERENCES "Aluno"("id") ON DELETE CASCADE,
  "vantagemId" TEXT NOT NULL REFERENCES "Vantagem"("id") ON DELETE CASCADE,
  "quantidade" INTEGER NOT NULL DEFAULT 1,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE ("alunoId", "vantagemId")
);

```

```
);
```

```
CREATE TABLE "Reconhecimento" (
  "id" TEXT PRIMARY KEY,
  "professorId" TEXT NOT NULL REFERENCES "Professor"("id") ON DELETE CASCADE,
  "alunoId" TEXT NOT NULL REFERENCES "Aluno"("id") ON DELETE CASCADE,
  "quantidade" INTEGER NOT NULL,
  "mensagem" TEXT NOT NULL,
  "tag" "TagReconhecimento" NOT NULL DEFAULT 'OUTRO',
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX "Reconhecimento_alunoId_createdAt_idx" ON "Reconhecimento"("alunoId",
"createdAt");
CREATE INDEX "Reconhecimento_professorId_createdAt_idx" ON
"Reconhecimento"("professorId", "createdAt");
```

```
CREATE TABLE "Transacao" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "tipo" "TipoTransacao" NOT NULL,
  "descricao" TEXT NOT NULL,
  "quantidadeMoedas" INTEGER NOT NULL,
  "referenciaId" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX "Transacao_usuarioId_createdAt_idx" ON "Transacao"("usuarioId",
"createdAt");
CREATE INDEX "Transacao_tipo_createdAt_idx" ON "Transacao"("tipo", "createdAt");
```

```
CREATE TABLE "Cupom" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "vantagemId" TEXT NOT NULL REFERENCES "Vantagem"("id") ON DELETE CASCADE,
  "codigo" TEXT UNIQUE NOT NULL,
  "custoMoedasSnapshot" INTEGER NOT NULL,
  "status" "StatusCupom" NOT NULL DEFAULT 'GERADO',
  "dataValidade" TIMESTAMP(3) NOT NULL,
  "dataUtilizacao" TIMESTAMP(3),
  "validadoPorEmpresaId" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL
);
CREATE INDEX "Cupom_status_dataValidade_idx" ON "Cupom"("status", "dataValidade");
CREATE INDEX "Cupom_usuarioId_status_idx" ON "Cupom"("usuarioId", "status");
```

```
CREATE TABLE "Notificacao" (
  "id" TEXT PRIMARY KEY,
  "usuarioId" TEXT NOT NULL REFERENCES "Usuario"("id") ON DELETE CASCADE,
  "titulo" TEXT NOT NULL,
  "mensagem" TEXT NOT NULL,
```



```

"lida"          BOOLEAN          NOT          NULL          DEFAULT          false,
"link"          TEXT,
"createdAt"     TIMESTAMP(3)     NOT          NULL          DEFAULT          CURRENT_TIMESTAMP
);
CREATE          INDEX            "Notificacao_usuarioId_lida_createdAt_idx"          ON
"Notificacao" ("usuarioId", "lida", "createdAt");

```

4.4 Estratégias de Integridade e Performance

4.4.1 Integridade Referencial

Ação	Comportamento Configurado
Exclusão de Usuario	ON DELETE CASCADE em Aluno, Professor, Empresa, Transacao, Cupom, Notificacao
Exclusão de Empresa	ON DELETE CASCADE em Vantagem (e, por cascata, em Cupons vinculados)
Exclusão de Aluno ou Professor	ON DELETE CASCADE em Reconhecimento
Exclusão de Instituicao	Restrito (não é possível excluir se há alunos/professores vinculados)
Exclusão de Categoria	Permitida; categoriaId em Vantagem fica NULL

4.4.2 Performance

- **Índices estratégicos** em colunas usadas em filtros e ordenações frequentes
- **Índices compostos** otimizam queries multi-coluna (ex: extrato cronológico)
- **Paginação** obrigatória em endpoints de listagem (limit/offset)
- **Conexão poolada** via Prisma (recomendado Prisma Accelerate em produção)
- **Caching** opcional em futuro para dashboards (Redis)

4.4.3 Segurança dos Dados

- **Senhas:** armazenadas apenas como hash bcrypt (salt rounds 10); nunca em texto plano
- **Dados sensíveis** (senhaHash): nunca retornados em responses da API
- **CPF/CNPJ:** exibidos mascarados na UI (111.***.***-11)
- **JWT Secret:** armazenado apenas em variável de ambiente
- **SSL:** conexão com banco PostgreSQL via TLS (padrão Neon)

4.5 Backup e Migrations

4.5.1 Migrations

- Gerenciadas pelo **Prisma Migrate** (prisma/migrations/)
- Versionadas no Git
- Aplicadas em produção via prisma migrate deploy
- Rollback manual (Prisma não oferece rollback automático em produção; estratégia é criar migrations de reversão)

4.5.2 Backup

- **Ambiente de produção (Neon):** backup automático gerenciado pelo provedor, com point-in-time recovery
- **Ambiente local:** pg_dump manual quando necessário
- **Dados críticos:** auditoria via Transacao serve como log imutável de movimentações

5. Códigos PlantUml

5.1 Diagrama de Casos de Uso

@startuml

title Diagrama de Casos de Uso - CoinPremier

actor Aluno

actor Professor

actor "Empresa Parceira" as Empresa

actor Admin

actor "Sistema (Cron)" as Sistema

rectangle CoinPremier {

' --- ALUNO ---

usecase "UC-04\nNavegar na Lojinha" as UC04

usecase "UC-05\nVer Detalhes da Vantagem" as UC05

usecase "UC-06\nFavoritar Vantagem" as UC06

usecase "UC-07\nAdicionar ao Carrinho" as UC07

usecase "UC-08\nResgatar Cupom" as UC08

usecase "UC-09\nVisualizar Meus Cupons" as UC09

usecase "UC-10\nConsultar Extrato" as UC10

usecase "UC-11\nVer Ranking Semestral" as UC11

usecase "UC-12\nVer Dashboard" as UC12

' --- SISTEMA AUTOMÁTICO ---

usecase "UC-20\nExpirar Cupons\nVencidos" as UC20sys

usecase "UC-29\nCreditar Moedas\nSemestralmente" as UC29

' --- PROFESSOR ---

usecase "UC-02\nCadastrar-se" as UC02

usecase "UC-24\nConsultar Extrato Prof" as UC24

usecase "UC-25\nVer Meus Alunos" as UC25

usecase "UC-13\nEnviar Moedas ao Aluno" as UC13

usecase "UC-43\nFazer Logout" as UC43

usecase "UC-06b\nVer Dashboard Prof" as UC06b

usecase "UC-08b\nNotificar Usuário" as UC08b

```

usecase "UC-41\nFazer Login" as UC41
usecase "UC-01\nVer Histórico de Resgates" as UC01
usecase "UC-22\nVer Dashboard Empresa" as UC22

```

```
' --- EMPRESA ---
```

```

usecase "UC-17\nCadastrar Vantagem" as UC17
usecase "UC-18\nEditar Vantagem" as UC18
usecase "UC-19\nRemover Vantagem" as UC19
usecase "UC-28\nVer Auditoria" as UC28
usecase "UC-20v\nValidar Cupom" as UC20v

```

```
' --- ADMIN ---
```

```

usecase "UC-23\nGerenciar Instituições" as UC23
usecase "UC-24a\nCadastrar Professor" as UC24a
usecase "UC-25a\nGerenciar Empresas" as UC25a
usecase "UC-26\nGerenciar Categorias" as UC26
usecase "UC-27\nVer Dashboard Admin" as UC27

```

```
' includes / extends
```

```

usecase "Escrever Mensagem" as incMsg
usecase "Validar Saldo" as incSaldo
usecase "Enviar Email de Cupom" as incEmail
usecase "Gerar Código de Cupom" as incCod
usecase "Debitar Saldo" as incDeb
usecase "Validar Credenciais" as incCred
usecase "Gerar Token JWT" as incToken
usecase "Upload de Imagem" as incUpload
usecase "Registrar Auditoria" as incAudit

```

```

UC13 ..> incMsg : <<include>>
UC13 ..> incSaldo : <<include>>
UC08 ..> incEmail : <<include>>
UC08 ..> incCod : <<include>>
UC08 ..> incDeb : <<include>>
UC41 ..> incCred : <<include>>
UC41 ..> incToken : <<include>>
UC17 ..> incUpload : <<include>>
UC20v ..> incAudit : <<include>>

```

```
}
```

```

Aluno --> UC04
Aluno --> UC05
Aluno --> UC06
Aluno --> UC07
Aluno --> UC08

```

Aluno --> UC09

Aluno --> UC10

Aluno --> UC11

Aluno --> UC12

Aluno --> UC02

Sistema --> UC20sys

Sistema --> UC29

Professor --> UC02

Professor --> UC24

Professor --> UC25

Professor --> UC13

Professor --> UC43

Professor --> UC06b

Professor --> UC41

Empresa --> UC41

Empresa --> UC01

Empresa --> UC22

Empresa --> UC17

Empresa --> UC18

Empresa --> UC19

Empresa --> UC28

Empresa --> UC20v

Admin --> UC23

Admin --> UC24a

Admin --> UC25a

Admin --> UC26

Admin --> UC27

Admin --> UC41

@enduml

5.2 DSS-01: Enviar Moedas (UC-13)

@startuml

title DSS - UC-13: Enviar Moedas ao Aluno

actor Professor

boundary Sistema

Professor -> Sistema : login(email, senha)

Sistema --> Professor : token JWT + dados

Professor -> Sistema : buscarAlunos(termo)

Sistema --> Professor : lista de alunos da instituição

Professor -> Sistema : enviarMoedas(alunoId, quantidade, tag, mensagem)

activate Sistema

Sistema -> Sistema : validarSaldoProfessor()

Sistema -> Sistema : debitarSaldoProfessor()

Sistema -> Sistema : creditarSaldoAluno()

Sistema -> Sistema : criarReconhecimento()

Sistema -> Sistema : criarTransacaoEnvio()

Sistema -> Sistema : criarTransacaoRecebimento()

Sistema -> Sistema : criarNotificacaoAluno()

Sistema -> Sistema : enviarEmailAoAluno()

deactivate Sistema

Sistema --> Professor : reconhecimento criado + saldo atualizado

@enduml

5.3 DSS-02: Resgatar Cupom (UC-08)

@startuml

title DSS - UC-08: Resgatar Cupom

actor Aluno

boundary Sistema

Aluno -> Sistema : login(email, senha)

Sistema --> Aluno : token JWT

Aluno -> Sistema : listarVantagens()

Sistema --> Aluno : vantagens disponíveis

Aluno -> Sistema : verDetalheVantagem(vantagemId)

Sistema --> Aluno : dados da vantagem

Aluno -> Sistema : resgatar(vantagemId)

activate Sistema

Sistema -> Sistema : validarSaldo()

Sistema -> Sistema : validarEstoque()

Sistema -> Sistema : validarLimitePorAluno()
Sistema -> Sistema : debitarSaldoAluno()
Sistema -> Sistema : decrementarEstoque()
Sistema -> Sistema : gerarCodigoCupom()
Sistema -> Sistema : criarCupom(status=GERADO)
Sistema -> Sistema : criarTransacaoResgate()
Sistema -> Sistema : enviarEmailCupomAoAluno()
Sistema -> Sistema : criarNotificacaoParaEmpresa()
deactivate Sistema

Sistema --> Aluno : cupom gerado (código + validade)

@enduml

5.4 DSS-03: Validar Cupom (UC-20)

@startuml

title DSS - UC-20: Validar Cupom

actor Empresa
boundary Sistema

Empresa -> Sistema : login(email, senha)
Sistema --> Empresa : token JWT

Empresa -> Sistema : buscarCupom(codigo)

activate Sistema
Sistema -> Sistema : validarCupomPertenceEmpresa()
Sistema -> Sistema : validarStatusCupom()
deactivate Sistema

Sistema --> Empresa : dados do cupom (aluno, vantagem, validade)

Empresa -> Sistema : confirmarValidacao(codigo)

activate Sistema
Sistema -> Sistema : atualizarStatusParaUTILIZADO()
Sistema -> Sistema : gravarDataUtilizacao()
Sistema -> Sistema : gravarValidadoPorEmpresaId()
Sistema -> Sistema : criarNotificacaoAluno()
Sistema -> Sistema : registrarAuditoria()
deactivate Sistema

Sistema --> Empresa : confirmação de validação

@enduml

5.5 Diagrama de Containers C4

@startuml

!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

title Diagrama de Containers (C4) - CoinPremier

Person(aluno, "Aluno", "Recebe moedas e resgata cupons")

Person(professor, "Professor", "Distribui moedas aos alunos")

Person(empresa, "Empresa", "Cadastra vantagens e valida cupons")

Person(admin, "Admin", "Gerencia o sistema")

System_Boundary(coinpremier, "CoinPremier") {

Container(frontend, "Frontend SPA", "React + Vite + Tailwind", "Interface de usuário responsiva")
 Container(backend, "API Backend", "Node.js + Express + Prisma", "API REST com regras de negócio")

Container(jobs, "Jobs Agendados", "node-cron", "Crédito semestral e expiração de cupons")

ContainerDb(db, "Banco de Dados", "PostgreSQL (Neon)", "Armazena usuários, moedas, cupons, etc.")

}

System_Ext(smtp, "SMTP Gmail", "Servidor de envio de emails")

Rel(aluno, frontend, "Acessa via navegador", "HTTPS")

Rel(professor, frontend, "Acessa via navegador", "HTTPS")

Rel(empresa, frontend, "Acessa via navegador", "HTTPS")

Rel(admin, frontend, "Acessa via navegador", "HTTPS")

Rel(frontend, backend, "Consome API", "JSON/HTTPS")

Rel(jobs, backend, "Dispara operações", "Interno")

Rel(backend, db, "Lê e escreve", "Prisma / SQL")

Rel(backend, smtp, "Envia emails", "SMTP")

@enduml

5.6 Diagrama de Componentes

@startuml

title Diagrama de Componentes - CoinPremier

package "Frontend (React + Vite)" {

```
component Pages
component "Components UI" as UI
component "Zustand Store" as Store
component "API Services\n(axios)" as APIService
component "React Router" as Router
```

```
Pages --> UI : usa
Pages --> Store : lê/escreve estado
Pages --> APIService : chama
Pages --> Router : navega
}
```

```
interface IRestAPI
```

```
APIService --> IRestAPI : requer
```

```
package "Backend (Node + Express)" {
  component Routes
  component "Middlewares\n(auth, validate, role)" as MW
  component Controllers
  component "Jobs\n(node-cron)" as Jobs
  component "Upload Service\n(Multer)" as Upload
  component "Services\n(regra de negócio)" as Services
  component "Email Service\n(Nodemailer)" as Email
  component Repositories
```

```
Routes --> MW
MW --> Controllers
Jobs --> Services : aciona
Controllers --> Upload : upload imagem
Controllers --> Services
Services --> Email : envia email
Services --> Repositories
}
```

```
interface IPrismaClient
```

```
interface ISMTP
```

```
Routes -- IRestAPI : provê
```

```
Repositories --> IPrismaClient : requer
```

```
component "Prisma Client" as Prisma
component "SMTP Gmail" as SMTPComp
```


Prisma -- IPismaClient : provê
 SMTPComp -- ISMTP : provê
 Email --> ISMTP : requer

database "PostgreSQL\n(Neon Cloud)" as DB
 Prisma --> DB : consulta/persiste

@enduml

5.7 Diagrama de Implantação

@startuml

title Diagrama de Implantação - CoinPremier

```
node "Dispositivo do Usuário" {
  component "Navegador Web\n(Chrome, Firefox, Safari)" as Browser
}
```

```
cloud "Vercel / Cloud Provider" {
  node "Servidor Web Estático" {
    artifact "Frontend Build\n(React SPA)" as SPA
  }
}
```

```
cloud "Cloud Provider\n(Render / Railway / AWS)" {
  node "Servidor Node.js" {
    artifact "Jobs Cron\n(node-cron)" as Cron
    artifact "API Backend\n(Express + Prisma)" as API
  }
  node "File Storage" {
    artifact "/uploads\n(imagens de vantagens)" as Files
  }
}
```

```
cloud "Neon Database" {
  database "PostgreSQL\ncoinpremier" as DB
}
```

```
cloud "Gmail SMTP" {
  component "Servidor SMTP" as SMTP
}
```

Browser --> SPA : HTTPS (carrega SPA)
 Browser --> API : HTTPS / REST API

Cron --> API : Dispara rotinas

API --> DB : Leitura/escrita local\n(Prisma / TCP 5432 SSL)

API --> Files : Leitura/escrita local

API --> SMTP : SMTP / porta 587

@enduml

5.7 Diagrama de Classes

@startuml

title Diagrama de Classes - CoinPremier

```
enum Role {  
    ALUNO  
    PROFESSOR  
    EMPRESA  
    ADMIN  
}
```

```
enum StatusUsuario {  
    ATIVO  
    BLOQUEADO  
}
```

```
enum TipoTransacao {  
    ENVIO  
    RECEBIMENTO  
    RESGATE  
    CREDITO_SEMESTRAL  
}
```

```
enum StatusCupom {  
    GERADO  
    UTILIZADO  
    EXPIRADO  
    CANCELADO  
}
```

```
enum TagReconhecimento {  
    PARTICIPACAO  
    CRIATIVIDADE  
    LIDERANCA  
    COLABORACAO  
    DEDICACAO  
    EXCELENCIA_ACADEMICA  
}
```

OUTRO

}

```
class Usuario {  
    +id : String  
    +nome : String  
    +email : String  
    +senhaHash : String  
    +role : Role  
    +status : StatusUsuario  
    +createdAt : DateTime  
    +updatedAt : DateTime  
    +autenticar() : boolean  
    +alterarSenha() : void  
}
```

```
class Instituicao {  
    +id : String  
    +nome : String  
    +sigla : String  
    +createdAt : DateTime  
    +gerenciarInstituicoes() : void  
    +cadastrarProfessor() : void  
}
```

```
class Aluno {  
    +id : String  
    +cpf : String  
    +rg : String  
    +endereco : String  
    +curso : String  
    +saldoMoedas : int  
    +resgatarVantagem() : Cupom  
    +adicionarFavorito() : void  
    +adicionarAoCarrinho() : void  
}
```

```
class Professor {  
    +id : String  
    +cpf : String  
    +departamento : String  
    +saldoMoedas : int  
    +ultimoSemestreCredito : String  
    +enviarMoedas() : Reconhecimento  
    +consultarExtrato() : List<Transacao>
```

```
}
```

```
class Empresa {  
    +id : String  
    +cnpj : String  
    +descricao : String  
    +cadastrarVantagem() : Vantagem  
    +validarCupom() : boolean  
}
```

```
class Admin {  
}
```

```
class Categoria {  
    +id : String  
    +nome : String  
    +slug : String  
    +icone : String  
}
```

```
class Vantagem {  
    +id : String  
    +titulo : String  
    +descricao : String  
    +custoMoedas : int  
    +foto : String  
    +estoque : int  
    +ativo : boolean  
    +validadeCupomDias : int  
    +limitePorAluno : int  
    +ativar() : void  
    +desativar() : void  
}
```

```
class Cupom {  
    +id : String  
    +codigo : String  
    +custoMoedasSnapshot : int  
    +status : StatusCupom  
    +dataValidade : DateTime  
    +dataUtilizacao : DateTime  
    +validadoPorEmpresaId : String  
    +gerarCodigo() : String  
    +marcarUtilizado() : void  
    +expirar() : void  
}
```

```
}
```

```
class Reconhecimento {  
  +id : String  
  +quantidade : int  
  +mensagem : String  
  +tag : TagReconhecimento  
  +createdAt : DateTime  
}
```

```
class Transacao {  
  +id : String  
  +tipo : TipoTransacao  
  +descricao : String  
  +quantidadeMoedas : int  
  +referenciaId : String  
  +createdAt : DateTime  
}
```

```
class Notificacao {  
  +id : String  
  +titulo : String  
  +descricao : String  
  +quantidadeMoedas : int  
  +lida : boolean  
  +link : String  
  +createdAt : DateTime  
}
```

```
class Favorito {  
  +id : String  
  +createdAt : DateTime  
}
```

```
class CarrinhoItem {  
  +id : String  
  +quantidade : int  
  +createdAt : DateTime  
  'Representa o ato de um professor enviar moedas a um aluno com mensagem.  
}
```

Usuario "1" -- "0..1" Aluno : registra

Usuario "1" -- "0..1" Professor

Usuario "1" -- "0..1" Empresa

Usuario "1" -- "0..1" Admin

Usuario "1" -- "0..*" Notificacao

Usuario "1" -- "0..*" Transacao

Aluno "0..*" -- "1" Instituicao : matrícula

Professor "0..*" -- "1" Instituicao

Empresa "1" -- "0..*" Vantagem : oferece

Categoria "1" -- "0..*" Vantagem : classifica

Aluno "1" -- "0..*" Favorito

Vantagem "1" -- "0..*" Favorito

Aluno "1" -- "0..*" CarrinhoItem

Vantagem "1" -- "0..*" CarrinhoItem

Professor "1" -- "0..*" Reconhecimento : faz

Aluno "1" -- "0..*" Reconhecimento : recebe

Aluno "1" -- "0..*" Cupom : resgata

Vantagem "1" -- "0..*" Cupom : referenciada

@enduml

5.8 DS-01: Sequência — UC-13 Enviar Moedas (detalhado)

@startuml

title Realização do Caso de Uso UC-13: Enviar Moedas ao Aluno

actor Professor

participant ProfessorController

participant ReconhecimentoService

participant ProfessorRepository

participant AlunoRepository

participant ReconhecimentoRepository

participant TransacaoRepository

participant NotificacaoService

participant EmailService

database PostgreSQL

Professor -> ProfessorController : POST /api/professor/reconhecimentos\n(alunoId, quantidade, tag, mensagem)

ProfessorController -> ReconhecimentoService : enviarReconhecimento(professorId, alunoId,\nquantidade, tag, mensagem)

ReconhecimentoService -> ProfessorRepository : buscarPorId(professorId)

ProfessorRepository -> PostgreSQL : SELECT * FROM Professor

PostgreSQL --> ProfessorRepository : professor

ProfessorRepository --> ReconhecimentoService : professor

ReconhecimentoService -> AlunoRepository : buscarPorId(alunoId)

AlunoRepository -> PostgreSQL : SELECT * FROM Aluno

PostgreSQL --> AlunoRepository : aluno

AlunoRepository --> ReconhecimentoService : aluno

ReconhecimentoService -> ReconhecimentoService : validarMesmaInstituicao(professor, aluno)

ReconhecimentoService -> ReconhecimentoService : validarSaldoSuficiente(professor, quantidade)

ReconhecimentoService -> ReconhecimentoService : validarMensagem(mensagem)

note over ReconhecimentoService : Inicia transação atômica\n(Prisma \$transaction)

ReconhecimentoService -> ProfessorRepository : decrementarSaldo(professorId, quantidade)

ProfessorRepository -> PostgreSQL : UPDATE Professor SET saldoMoedas

ReconhecimentoService -> AlunoRepository : incrementarSaldo(alunoId, quantidade)

AlunoRepository -> PostgreSQL : UPDATE Aluno SET saldoMoedas

ReconhecimentoService -> ReconhecimentoRepository : criar(professorId, alunoId, quantidade, tag, mensagem)

ReconhecimentoRepository -> PostgreSQL : INSERT INTO Reconhecimento

PostgreSQL --> ReconhecimentoRepository : reconhecimento

ReconhecimentoRepository --> ReconhecimentoService : reconhecimento

ReconhecimentoService -> TransacaoRepository : criar(tipo=ENVIO, professor)

TransacaoRepository -> PostgreSQL : INSERT INTO Transacao

ReconhecimentoService -> TransacaoRepository : criar(tipo=RECEBIMENTO, aluno)

TransacaoRepository -> PostgreSQL : INSERT INTO Transacao

ReconhecimentoService -> NotificacaoService : notificarAluno(alunoId, reconhecimento)

NotificacaoService -> PostgreSQL : INSERT INTO Notificacao

ReconhecimentoService -> EmailService : enviarReconhecimentoRecebido(aluno, dados)

EmailService --> ReconhecimentoService : emailEnviado

note over ReconhecimentoService : Commit da transação

ReconhecimentoService --> ProfessorController : {reconhecimento, saldoAtualizado}

ProfessorController --> Professor : 201 Created + dados

@enduml

5.9 DS-02: Sequência — UC-08 Resgatar Cupom (detalhado)

@startuml

title Realização do Caso de Uso UC-08: Resgatar Cupom

actor Aluno
 participant AlunoController
 participant ResgateService
 participant AlunoRepository
 participant VantagemRepository
 participant CupomRepository
 participant TransacaoRepository
 participant EmailService
 participant NotificacaoService
 database PostgreSQL

Aluno -> AlunoController : POST /api/aluno/resgatar\n(vantagemId)
 AlunoController -> ResgateService : resgatar(alunoId, vantagemId)

ResgateService -> AlunoRepository : buscarPorId(alunoId)
 AlunoRepository -> PostgreSQL : SELECT * FROM Aluno
 PostgreSQL --> AlunoRepository : aluno
 AlunoRepository --> ResgateService : aluno

ResgateService -> VantagemRepository : buscarPorId(vantagemId)
 VantagemRepository -> PostgreSQL : SELECT * FROM Vantagem
 PostgreSQL --> VantagemRepository : vantagem
 VantagemRepository --> ResgateService : vantagem

ResgateService -> ResgateService : validarSaldoSuficiente()
 ResgateService -> ResgateService : validarEstoque()
 ResgateService -> ResgateService : validarLimitePorAluno()
 ResgateService -> ResgateService : validarAtivo()

note over ResgateService : Inicia transação atômica\n(Prisma \$transaction)

ResgateService -> AlunoRepository : decrementarSaldo(alunoId, custoMoedas)
 AlunoRepository -> PostgreSQL : UPDATE Aluno SET saldoMoedas

ResgateService -> VantagemRepository : decrementarEstoque(vantagemId)
 VantagemRepository -> PostgreSQL : UPDATE Vantagem SET estoque

ResgateService -> ResgateService : gerarCodigoCupom()

note right : Formato: RSG-XXXXXX

ResgateService -> CupomRepository : criar(codigo, alunoId, vantagemId, \ncustoSnapshot, dataValidade, status=GERADO)

CupomRepository -> PostgreSQL : INSERT INTO Cupom

PostgreSQL --> CupomRepository : cupom

CupomRepository --> ResgateService : cupom

ResgateService -> TransacaoRepository : criar(tipo=RESGATE, aluno, quantidade)

TransacaoRepository -> PostgreSQL : INSERT INTO Transacao

ResgateService -> EmailService : enviarCupomAoAluno(aluno, cupom)

EmailService --> ResgateService : emailEnviado

ResgateService -> NotificacaoService : notificarEmpresa(empresaId, cupom)

NotificacaoService -> PostgreSQL : INSERT INTO Notificacao

note over ResgateService : Commit da transação

ResgateService --> AlunoController : {cupom, saldoAtualizado}

AlunoController --> Aluno : 201 Created + dados do cupom

@enduml

5.10 DS-04: Sequência — UC-02 Cadastrar Aluno (detalhado)

@startuml

title Realização do Caso de Uso UC-02: Cadastrar-se (Aluno)

actor Aluno

participant AuthController

participant AuthService

participant UsuarioRepository

participant AlunoRepository

participant InstituicaoRepository

database PostgreSQL

Aluno -> AuthController : POST /api/auth/cadastro/aluno\n{nome, email, cpf, rg, ...}

AuthController -> AuthController : validarDadosZod(body)

AuthController -> AuthService : cadastrarAluno(dados)

AuthService -> UsuarioRepository : buscarPorEmail(email)

UsuarioRepository -> PostgreSQL : SELECT FROM Usuario

PostgreSQL --> UsuarioRepository : null

UsuarioRepository --> AuthService : null

AuthService -> AlunoRepository : buscarPorCpf(cpf)
 AlunoRepository -> PostgreSQL : SELECT FROM Aluno
 PostgreSQL --> AlunoRepository : null
 AlunoRepository --> AuthService : null

AuthService -> InstituicaoRepository : buscarPorId(instituicaoId)
 InstituicaoRepository -> PostgreSQL : SELECT FROM Instituicao
 PostgreSQL --> InstituicaoRepository : instituicao
 InstituicaoRepository --> AuthService : instituicao

AuthService -> AuthService : gerarHashSenha(senha)

note over AuthService : Inicia transação atômica\n(Prisma \$transaction)

AuthService -> UsuarioRepository : criar({nome, email, senhaHash,\nrole: ALUNO, status: ATIVO})
 UsuarioRepository -> PostgreSQL : INSERT INTO Usuario
 PostgreSQL --> UsuarioRepository : usuario
 UsuarioRepository --> AuthService : usuario

AuthService -> AlunoRepository : criar({usuarioId, cpf, rg, endereco,\ncurso, instituicaoId, saldoMoedas: 0})
 AlunoRepository -> PostgreSQL : INSERT INTO Aluno
 PostgreSQL --> AlunoRepository : aluno
 AlunoRepository --> AuthService : aluno

note over AuthService : Commit da transação

AuthService -> AuthService : gerarTokenJWT(usuario)
 AuthService --> AuthController : {token, usuario}
 AuthController --> Aluno : 201 Created + token JWT

@enduml

5.11 DC-01: Comunicação — UC-13 Enviar Moedas

@startuml

title Diagrama de Comunicação - UC-13 Enviar Moedas

object ":Professor" as Prof
 object ":ProfessorController" as PC
 object ":ReconhecimentoService" as RS

```
object ":ProfessorRepo" as PR
object ":AlunoRepo" as AR
object ":ReconhecimentoRepo" as RR
object ":TransacaoRepo" as TR
object ":NotificacaoService" as NS
object ":EmailService" as ES
```

```
Prof --> PC : 1: enviarReconhecimento()
PC --> RS : 2: executar()
RS --> RS : 5: validarRegras()
RS --> PR : 3: buscarProfessor()
RS --> PR : 6: decrementarSaldo()
RS --> AR : 4: buscarAluno()
RS --> AR : 7: incrementarSaldo()
RS --> RR : 8: criarReconhecimento()
RS --> TR : 9: criarTransacaoEnvio()
RS --> TR : 10: criarTransacaoRecebimento()
RS --> NS : 11: notificarAluno()
RS --> ES : 12: enviarEmail()
```

@enduml

5.12 DC-02: Comunicação — UC-08 Resgatar Cupom

@startuml

title Diagrama de Comunicação - UC-08 Resgatar Cupom

```
object ":Aluno" as A
object ":AlunoController" as AC
object ":ResgateService" as RS
object ":VantagemRepo" as VR
object ":AlunoRepo" as AR
object ":CupomRepo" as CR
object ":TransacaoRepo" as TR
object ":EmailService" as ES
object ":NotificacaoService" as NS
```

```
A --> AC : 1: resgatar(vantagemId)
AC --> RS : 2: executar()
RS --> RS : 5: validarSaldo()
RS --> RS : 6: validarEstoque()
RS --> RS : 7: validarLimitePorAluno()
RS --> RS : 10: gerarCodigoCupom()
RS --> VR : 4: buscarVantagem()
```

```
RS --> VR : 9: decrementarEstoque()  
RS --> AR : 3: buscarAluno()  
RS --> AR : 8: decrementarSaldo()  
RS --> CR : 11: criarCupom()  
RS --> TR : 12: criarTransacaoResgate()  
RS --> ES : 13: enviarEmailAluno()  
RS --> NS : 14: notificarEmpresa()
```

@enduml

5.13 DC-03: Comunicação — UC-20 Validar Cupom

@startuml

title Diagrama de Comunicação - UC-20 Validar Cupom

```
object ":Empresa" as E  
object ":EmpresaController" as EC  
object ":CupomService" as CS  
object ":CupomRepo" as CR  
object ":NotificacaoService" as NS  
object ":AuditService" as AS
```

```
E --> EC : 1: buscarCupom(codigo)  
E --> EC : 7: confirmarValidacao(codigo)  
EC <-- E : 6: exibirCupom
```

```
EC --> CS : 2: buscar()  
EC --> CS : 8: validar()  
CS --> E : 5: retornarDados
```

```
CS --> CS : 4: validarPropriedadeEmpresa()  
CS --> CS : 10: validarStatus()  
CS --> CS : 11: validarValidade()
```

```
CR <-- CS : 3: buscarPorCodigo()  
CR <-- CS : 9: buscarPorCodigo()  
CR <-- CS : 12: atualizarParaUTILIZADO()
```

```
NS <-- CS : 13: notificarAluno()  
AS <-- CS : 14: registrarAuditoria()
```

@enduml

5.14 DC-04: Comunicação — UC-02 Cadastrar Aluno

@startuml

title Diagrama de Comunicação - UC-02 Cadastrar-se (Aluno)

```

object ":Aluno" as A
object ":AuthController" as AC
object ":AuthService" as AS
object ":UsuarioRepo" as UR
object ":AlunoRepo" as AR
object ":InstituicaoRepo" as IR

```

```

A --> AC : 1: cadastrarAluno(dados)
A <-- AC : 12: respondeSucesso

```

```

AC --> AC : 2: validarZod()
AC --> AS : 3: executar()
AS --> AC : 11: retornarToken

```

```

AS --> AS : 7: gerarHashSenha()
AS --> AS : 10: gerarTokenJWT()

```

```

UR <-- AS : 4: verificarEmailUnico()
UR <-- AS : 8: criarUsuario()

```

```

AR <-- AS : 5: verificarCpfUnico()
AR <-- AS : 9: criarAluno()

```

```

IR <-- AS : 6: verificarInstituicao()

```

@enduml

5.15 DE-01: Estados do Cupom

@startuml

title Diagrama de Estados - Cupom

```

[*] --> GERADO : resgatarVantagem()\n[saldo suficiente]\n[estoque disponível]

```

```

state GERADO {
    entry / gerarCodigo()
    entry / definirValidade()
    entry / enviarEmailAluno()
    note right : Estado inicial.\nCupom aguardando uso\nna empresa parceira.
}

```

GERADO --> UTILIZADO : validarCupom()\n[empresa dona da vantagem]\n[dentro da validade]
 GERADO --> EXPIRADO : cronExpiraCupons()\n[dataValidade < now]
 GERADO --> CANCELADO : cancelarCupom()\n[ação administrativa]

```
state UTILIZADO {
  entry / gravarDataUtilizacao()
  entry / gravarValidadoPorEmpresaId()
  entry / notificarAluno()
  note right : Cupom usado presencialmente.\nEmpresa confirmou o uso.
}
```

```
state EXPIRADO {
  note right : Ultrapassou data de validade.\nNão pode mais ser usado.
}
```

```
state CANCELADO {
  note right : Cancelado manualmente\n(casos excepcionais).
}
```

UTILIZADO --> [*]
 EXPIRADO --> [*]
 CANCELADO --> [*]

@enduml

5.16 DE-02: Estados do Usuário

@startuml

title Diagrama de Estados - Usuário

[*] --> ATIVO : cadastrar()\n(aluno, empresa)\nou criar()\n(professor, admin)

```
state ATIVO {
  entry / permitirLogin()
  entry / habilitarOperacoes()
  note right : Usuário pode fazer login\ne utilizar todas as\ntotalidades do seu perfil.
}
```

ATIVO --> BLOQUEADO : bloquear()\n[ação do admin]
 BLOQUEADO --> ATIVO : desbloquear()\n[ação do admin]

```
state BLOQUEADO {
  entry / invalidarSessoes()
```

```

    entry / negarLogin()
    note right : Login bloqueado.\nDados preservados\npara auditoria.
}

```

```

ATIVO --> [*] : excluirConta()\n(hard delete)
BLOQUEADO --> [*] : excluirConta()

```

```
@enduml
```

5.17 DE-03: Estados da Vantagem

```

@startuml
title Diagrama de Estados - Vantagem

```

```

[*] --> ATIVA : cadastrar()\n[empresa autenticada]

```

```

state ATIVA {
    entry / exibirNaLoja()
    entry / permitirResgate()
    note right : Visível na loja.\nAlunos podem resgatar.
}

```

```

ATIVA --> ATIVA_SEM_ESTOQUE : decrementarEstoque()\n[estoque = 0]
ATIVA_SEM_ESTOQUE --> ATIVA : reporEstoque()\n[estoque > 0]

```

```

state ATIVA_SEM_ESTOQUE {
    entry / exibirComoEsgotado()
    entry / bloquearResgates()
    note right : Visível mas esgotada.\nBotão "Resgatar" desabilitado.
}

```

```

ATIVA --> INATIVA : desativar()\n[empresa]
ATIVA_SEM_ESTOQUE --> INATIVA : desativar()
INATIVA --> ATIVA : ativar()\n[se estoque > 0]

```

```

state INATIVA {
    entry / ocultarDaLoja()
    entry / bloquearNovosResgates()
    note right : Removida da loja.\nCupons já emitidos\ncontinuum válidos.
}

```

```

ATIVA --> [*] : remover()\n(soft delete: ativo = false)
INATIVA --> [*] : remover()

```

@enduml

5.18 DE-04: Estados do Reconhecimento

@startuml

title Diagrama de Estados - Reconhecimento

[*] --> VALIDACAO : enviarReconhecimento()

```
state VALIDACAO {
  entry / validarSaldo()
  entry / validarInstituicao()
  entry / validarMensagem()
}
```

VALIDACAO --> REGISTRADO : validacoesOk()

VALIDACAO --> [*] : validacaoFalhou()\n[erro exibido]

```
state REGISTRADO {
  entry / debitarProfessor()
  entry / creditarAluno()
  entry / criarTransacoes()
  entry / notificarAluno()
  entry / enviarEmail()
  note right : Estado final.\nReconhecimento registrado\npermanentemente no sistema.\nAluno notificado.
}
```

REGISTRADO --> [*]

@enduml

5.19 DE-05: Estados da Notificação

@startuml

title Diagrama de Estados - Notificação

[*] --> NAO_LIDA : criar()\n[evento do sistema]

```
state NAO_LIDA {
  entry / incrementarBadge()
  entry / exibirDestacada()
  note right : Exibida com destaque\n(fundo indigo-50 + bolinha azul).\nContador do sininho atualizado.
```



```
}

```

```
NAO_LIDA --> LIDA : marcarComoLida()\n[clique do usuário]
LIDA --> NAO_LIDA : marcarComoNaoLida()\n[ação reversível]

```

```
state LIDA {
  entry / decrementarBadge()
  note right : Sem destaque visual.\nContador do sininho atualizado.
}

```

```
NAO_LIDA --> [*] : excluir()\n[usuário apaga]
LIDA --> [*] : excluir()
LIDA --> [*] : limpezaAutomatica()\n[> 90 dias, via cron]

```

```
@enduml

```

5.20 DE-06: Estados do Item do Carrinho

```
@startuml

```

```
title Diagrama de Estados - Item do Carrinho

```

```
[*] --> NO_CARRINHO : adicionarAoCarrinho()

```

```
state NO_CARRINHO {
  entry / registrarItem()
  entry / atualizarResumo()
  note right : Item persistido no banco.\nQuantidade ajustável.
}

```

```
NO_CARRINHO --> NO_CARRINHO : alterarQuantidade()\n[respeitando estoque\ne limitePorAluno]

```

```
NO_CARRINHO --> FINALIZADO : finalizarResgate()\n[saldo suficiente para todos os itens]
NO_CARRINHO --> [*] : removerItem()\n[ação do aluno]

```

```
state FINALIZADO {
  entry / gerarCupomPorItem()
  entry / debitarSaldoTotal()
  entry / limparCarrinho()
  note right : Resgate efetuado.\nItem sai do carrinho\ne vira Cupom(s).
}

```

```
FINALIZADO --> [*]

```

@enduml